

目标检测中常见指标

TP (True Positive, 真正例): IoU (交并比 >0.5 的检测框数量 (同一真实框仅计算一次)。

FP (False Positive, 假正例): IoU ≤ 0.5 的检测框数量, 或检测到同一个真实框的多余检测框数量。

FN (False Negative, 假负例): 未被检测到的真实框数量。

Precision (精确率/查准率): $TP / (TP + FP)$ 。即在模型预测出的所有目标中, 预测正确的比例。

Recall (召回率/查全率): $TP / (TP + FN)$ 。即在所有真实目标中, 被模型正确预测出来的比例。

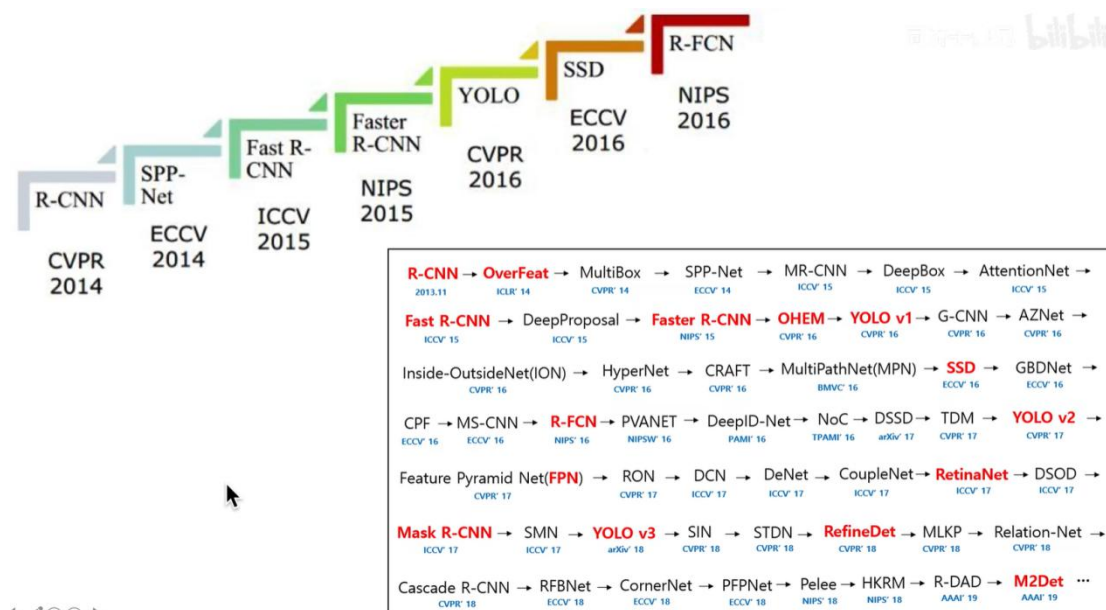
P-R 曲线 (Precision-Recall 曲线): 以召回率为横轴、精确率为纵轴绘制的曲线。

AP (Average Precision, 平均精度): P-R 曲线与坐标轴所围成的面积。

mAP (mean Average Precision, 平均精度均值): 所有类别的 AP 值的平均值, 是衡量模型整体性能的核心指标。

训练集、验证集与测试集: 比如学校要从学生里选拔一群人去参加市里的学科竞赛, 每个学生要去刷题(训练数据集), 一段时间后学校来组织测试(验证数据集), 根据成绩高低来确定参加竞赛的人选, 确定人选后安排去市里面参加

竞赛(测试数据集)。训练集和验证集来对模型进行调优。



如果标签是离散值，则为分类；如猫狗图像分类和手写数字识别。

如果标签是连续值，则为回归；如股价、房价、用电量预测。

减少过拟合的方法：数据增强、Dropout。

底层网络提取边缘、特色等特征；中层网络提取形状特征；高层网络提取物体特征。

常用的分类模型：决策树、朴素贝叶斯、支持向量机、KNN、神经网络逻辑回归、线性判别分析、随机森林、隐马尔可夫、贝叶斯网络。

二分类混淆矩阵 Confusion Matrix

真实 \ 预测	预测为肺炎 Positive	预测为健康 Negative
肺炎 Actual Positive	True Positive ✓	False Negative ✗
健康 Actual Negative	False Positive ✗	True Negative ✓

✓: 正确分类 True
✗: 错误分类 False

二分类混淆矩阵算例

n=100	预测为肺炎	预测为健康
真实肺炎	45	5
真实健康	15	35

TP=45 FN=5
FP=15 TN=35

$$\begin{aligned} \text{Precision} &= \frac{45}{45+15} = 0.75 \quad \text{查准率} \\ \text{Recall} &= \frac{45}{45+5} = 0.9 \quad \text{查全率 敏感性 召回率} \\ \text{F1-score} &= \frac{2 \times 0.75 \times 0.9}{0.75 + 0.9} = 0.82 \\ \text{Accuracy} &= \frac{45+35}{100} = 80\% \quad \text{准确率} \\ \text{Specificity} &= \frac{35}{15+35} = 0.7 \quad \text{特异性} \end{aligned}$$

混淆矩阵：一个 2 行 2 列的矩阵。

TP：真正例 (True Positive)。实际为正例，预测也为正例的个数。

FN：假负例 (False Negative)。实际为正例，但预测为负例的个数。（漏报）

FP：假正例 (False Positive)。实际为负例，但预测为正例的个数。（误报）

TN：真负例 (True Negative)。实际为负例，预测也为负例的个数。

重要性和关系：

TP + FN：真实为正例（如：肺炎）的数据总个数。

FP + TN: 真实为负例（如：健康）的数据总个数。

TP + TN: 所有被正确分类的数据总个数。

核心评估指标（#表示个数）

1. 准确率

公式: $\text{Accuracy} = \frac{\text{正确分类}}{\text{所有数据}}$

$$= \frac{TP+TN}{TP+TN+FP+FN}$$

释义: 所有预测中，预测正确的比例。

2. 精确率/查准率

公式: $\text{Precision} = \frac{TP}{TP+FP}$

释义: 在所有预测为正例的数据中，真正是正例的比例。

通俗解释: “预测为肺炎的数据中，有多少真的是肺炎”。

3. 召回率/查全率/敏感性

公式: $\text{Recall} = \frac{TP}{TP + FN}$

释义: 在所有真实为正例的数据中，被正确预测出来的比例。

通俗解释: “真实肺炎数据中，有多少被预测出来了”。

4. F1 Score

公式: $\text{F1 Score} =$

$$2 * \text{Precision} * \text{Recall} / (\text{Precision} + \text{Recall})$$

释义: 精确率和召回率的调和平均数，用于综合反映分类器在这两个指标上的表现。

5. 特异性

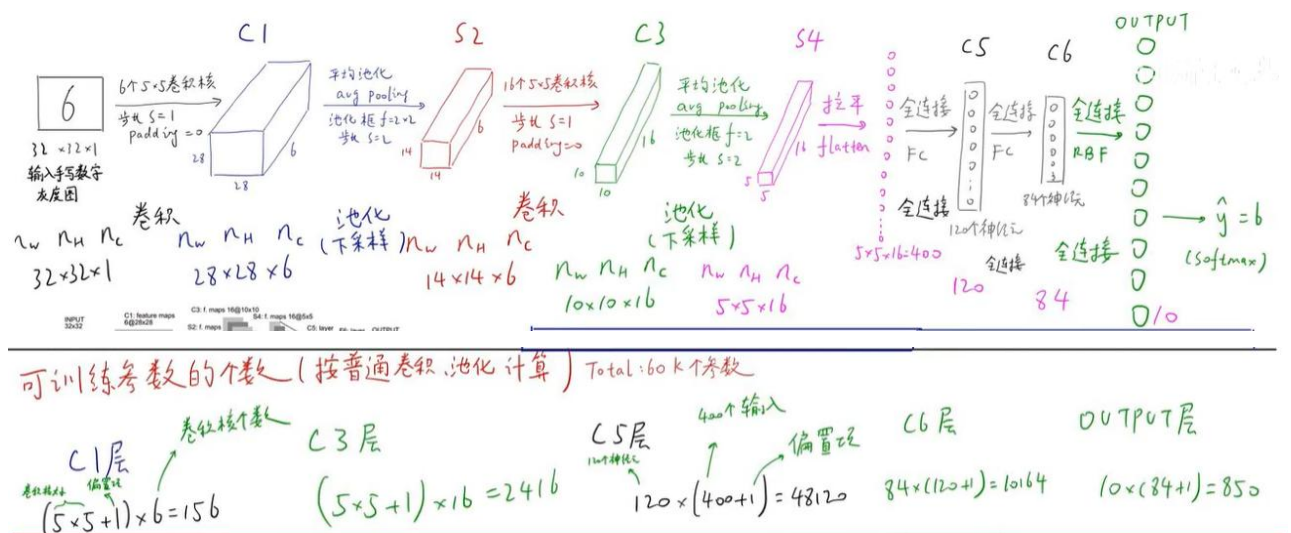
公式: $\text{Specificity} = \text{TN} / (\text{FP} + \text{TN})$

释义: 在所有真实为负例的数据中, 被正确预测为负例的比例。

通俗解释: “真健康数据中, 有多少被正确识别为健康”。

池化的作用: 减少参数量; 防止过拟合。

LeNet:



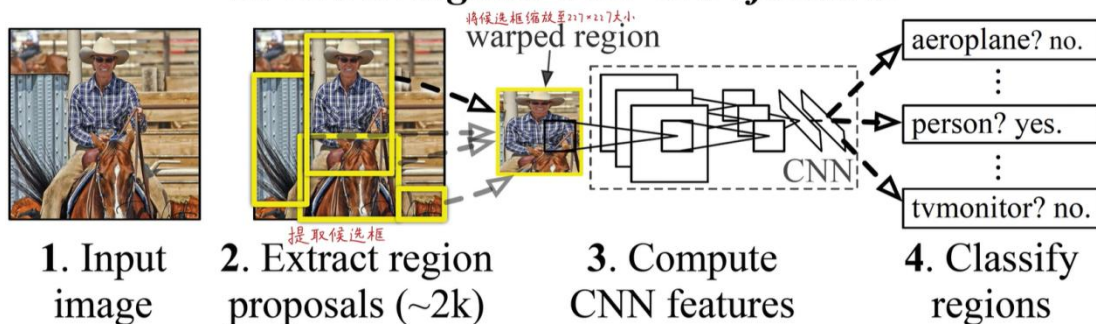
本论文★

两阶段(多阶段)目标检测演进

RCNN	SPPNet	Fast RCNN	Faster RCNN	FPN	Mask RCNN
2013.11.11	2014	2015.4.30	2015.6.4	2016.12.9	2017.3.20
CVPR 2014	ECCV 2014	ICCV 2015	NIPS 2015	CVPR 2017	ICCV 2017
CNN目标检测	空间金字塔池化	ROI-Pooling	RPN Anchor	特征金字塔	实例分割

R-CNN:

R-CNN: Regions with CNN features

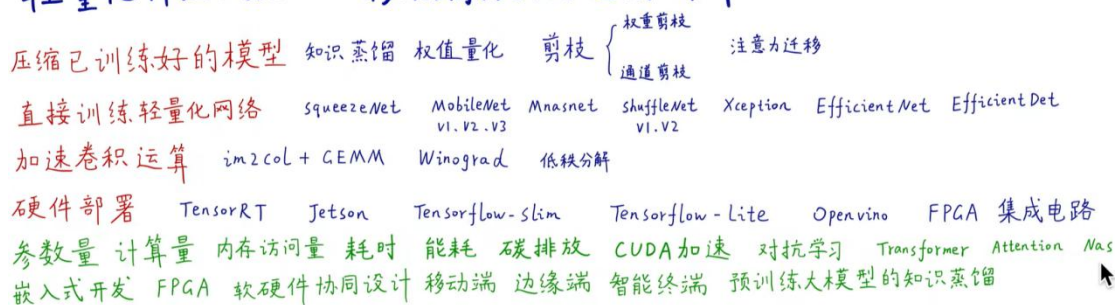


改进 R-CNN

- 提取候选框: EdgeBoxes、RPN 网络
- 共享卷积运算: SPPNet、Fast R CNN
- 兼容任意尺寸图像: SPP、ROI Pooling
- 预设长宽比: Anchor
- 网络结构: 端到端
- 融合各层特征: FPN

MobileNet V1:

轻量化神经网络 移动端、终端边缘计算



深度可分离卷积:

深度可分离卷积是一种将标准卷积操作分解为两步的、计算效率极高的替代方案。它的核心目标是在基本不影响模

型表达能力的前提下，极大地减少计算成本和参数量。

想象一下，您有一张 RGB 彩色图片（3 个通道：红、绿、蓝），并且想用一组过滤器（卷积核）来提取特征。

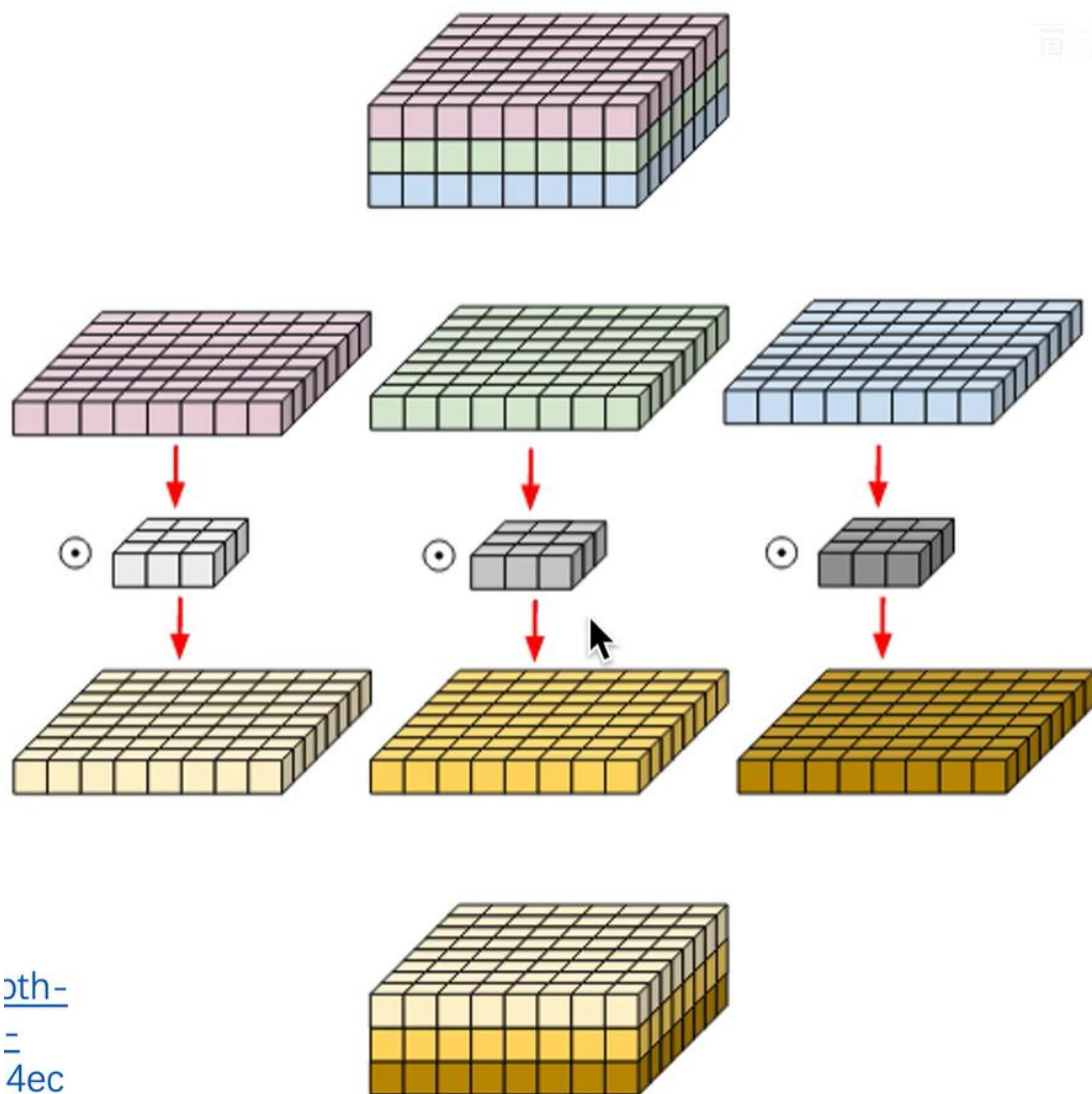
1. 传统卷积 (Standard Convolution)

- 过程：每个过滤器都必须“同时”查看输入的所有 3 个通道，并计算出一个输出值。如果想得到 256 个特征图，就需要 256 个这样的过滤器。

- 问题：计算是“全连接”的，每个过滤器和所有输入通道都要做密集的乘法运算，计算量巨大。

- 你可以把它想象成：一个多面手团队，团队里的每个专家（一个过滤器）都必须精通所有领域（所有输入通道），才能完成工作。这需要大量的培训和协作成本（计算量）。

2. 深度可分离卷积 (Depthwise Separable Convolution)



它将上述过程分解为两个独立的、更简单的步骤：

第一步：深度卷积

- 过程：我们不再使用一个“全才”过滤器。相反，我们准备 3 个“专才”过滤器（数量等于输入通道数），每个过滤器只负责查看一个输入通道。

- 红色过滤器只看红色通道，输出 1 个特征图。

- 绿色过滤器只看绿色通道，输出 1 个特征图。

- 蓝色过滤器只看蓝色通道，输出 1 个特征图。

- 结果：我们将这 3 个结果堆叠起来，得到一个 3 通道的“中间特征图”。这一步只做空间滤波（检测边缘、纹理等），没有进行通道间的信息融合。

- 类比：现在有三个专项顾问，各自只处理自己最擅长的单一领域。效率高，但他们的成果是独立的，没有整合。

第二步：逐点卷积

- 过程：为了解决信息未融合的问题，我们使用一个 1×1 的常规卷积。这个卷积核的大小是 $1 \times 1 \times 3$ （因为上一步输出的中间特征图有 3 个通道）。

- 这个 1×1 卷积会同时查看上一步得到的 3 个通道，并对它们进行加权组合，生成一个新的单通道特征图。

- 如果我们想要 256 个最终输出通道，我们就用 256 个这样的 1×1 卷积核。

- 结果： 1×1 卷积高效地完成了通道融合的任务，将上一步的独立信息整合起来，并可以自由地升维或降维。

- 类比：现在来了一位项目经理（ 1×1 卷积），他的工作就是听取三位专项顾问的报告，把他们独立的成果整合成一份完整的方案（一个融合的特征）。如果需要 256 份不同侧重的方案，就派 256 个项目经理。

为什么它能大幅降低计算量？（数学直观）

标准卷积乘法计算量 $(D_K \cdot D_K \cdot M) \cdot (N \cdot D_F \cdot D_F)$

一次卷积的乘法次数 $D_K \cdot D_K \cdot M \cdot N$

输出 feature map 元素个数

Depthwise separable convolutions cost:

Depthwise 卷积 $(D_K \cdot D_K) \cdot (M \cdot D_F \cdot D_F)$

pointwise 卷积 $M \cdot (N \cdot D_F \cdot D_F)$

一次卷积的乘法次数 $D_K \cdot D_K \cdot M \cdot D_F \cdot D_F$

输出 feature map 元素个数 $M \cdot D_F \cdot D_F$

一次 1×1 卷积的乘法次数 $M \cdot N \cdot D_F \cdot D_F$

输出 feature map 元素个数 $M \cdot N \cdot D_F \cdot D_F$

(5)

which is the sum of the depthwise and 1×1 pointwise convolutions.

参数量 $D_K \cdot D_K \cdot M + 1 \cdot 1 \cdot M \cdot N$

相比于(5)式, 所有 M, N 乘 α , 所有 D_F 乘 ρ

$$D_K \cdot D_K \cdot \alpha M \cdot \rho D_F \cdot \rho D_F + \alpha M \cdot \alpha N \cdot \rho D_F \cdot \rho D_F$$

计算量 $\frac{D_K \cdot D_K \cdot M \cdot D_F \cdot D_F + M \cdot N \cdot D_F \cdot D_F}{D_K \cdot D_K \cdot M \cdot N \cdot D_F \cdot D_F}$

$$= \frac{1}{N} + \frac{1}{D_K^2}$$

N : 输出通道数 (point wise 卷积核个数)

D_K : depthwise 卷积核尺寸

参数量 $\frac{D_K \cdot D_K \cdot M + M \cdot N}{D_K \cdot D_K \cdot M \cdot N} = \frac{1}{N} + \frac{1}{D_K^2}$

假设输入特征图尺寸为 $D_F \times D_F \times M$ (例如, $14 \times 14 \times 256$), 我们想用 N 个大小为 $D_K \times D_K$ (例如, 3×3) 的卷积核, 输出尺寸为 $D_F \times D_F \times N$ 。

• 标准卷积计算成本: $D_K * D_K * M * N * D_F * D_F$

(每个卷积核的每个参数都要和输入的每个通道在空

间上做计算)

•深度可分离卷积计算成本:

1. 深度卷积成本: $D_K * D_K * M * 1 * D_F * D_F$

(M 个单通道滤波器)

2. 逐点卷积成本: $1 * 1 * M * N * D_F * D_F$

(N 个 1x1 的滤波器, 作用于 M 个通道)

总成本=深度成本+逐点成本

计算量对比:

深度可分离卷积的计算量/标准卷积的计算量

$= 1/N + 1/(D_K^2)$

以常用 3x3 卷积和输出通道数 N=256 为例:

比例 $= 1/256 + 1/9 \approx 0.115$

这意味着, 深度可分离卷积的理论计算量只有标准卷积的约 11.5%! 参数量也同比大幅下降。

总结: 优缺点与应用

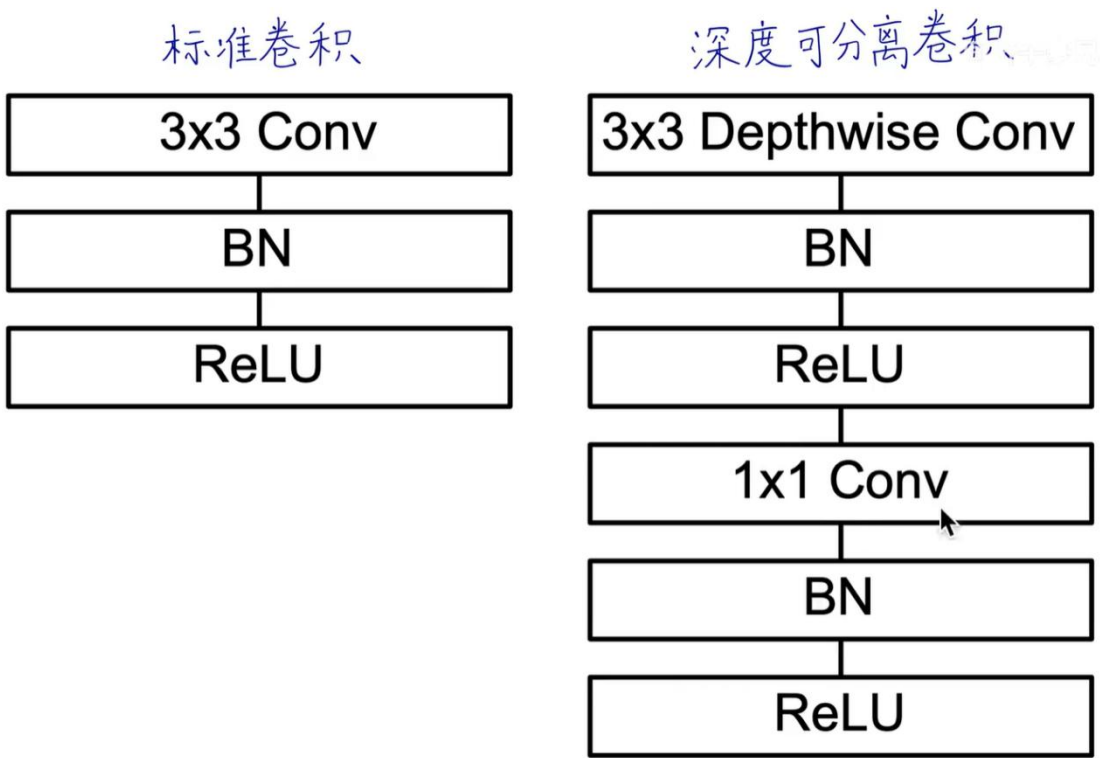
特性	深度可分离卷积
核心思想	解耦: 将标准卷积的空间滤波和通道融合两个任务分离执行。
最大优势	极高的计算效率和参数效率, 非常适合移动端、嵌入式设备 (如您的深山搜救无人机)。
潜在缺点	由于分解操作, 其特征提取能力可能略弱于在相同参数量下精心设计的标准卷积块。但对于绝大多数任务, 其效率优势远超微小的精度损失。
著名应用	MobileNet系列的核心构建块, 正是凭借它成为移动端视觉任务的标杆。Xception 网络则将其思想发挥到极致。

对您项目的意义:

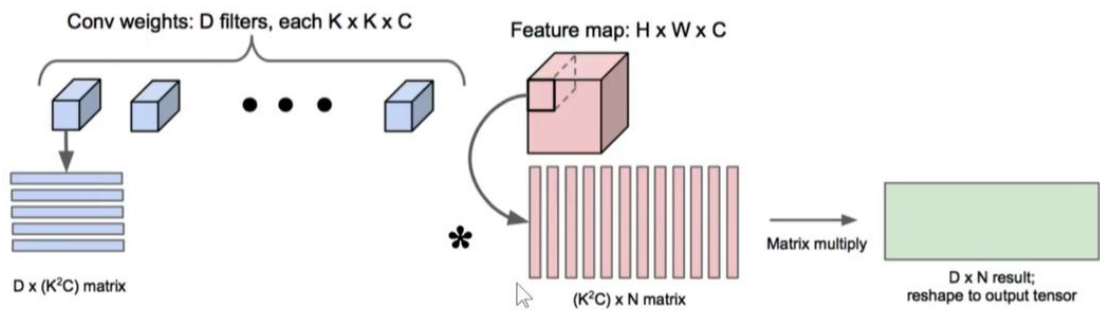
在您的《深山搜救多模态检测系统》中, 若要实现申报书中强调的“轻量化”和“边缘计算”, 深度可分离卷积是构建模型骨干网络的首选技术之一。您可以在设计多模态融

合网络时，用深度可分离卷积块替代大部分标准卷积，从而确保模型在无人机搭载的 Jetson 等边缘设备上，能够实现高帧率、低功耗的实时检测，这是项目从理论走向实际应用的关键一步。

MobileNet V1 网络结构：



把卷积运算转成了矩阵运算，而矩阵运算是可以加速的。



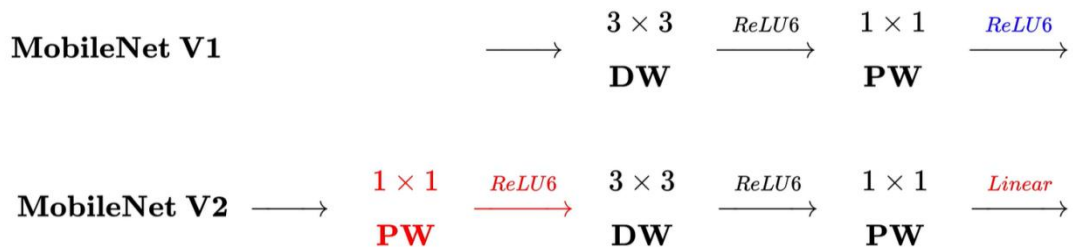
MobileNet V2:

MobileNet V1的局限:

1. 没有残差连接

2. 很多Depthwise卷积核训练出来是0

卷积核 权重数量和通道数量太少,太“单薄”
ReLU
低精度



这张图非常清晰地展示了 MobileNet 从 V1 到 V2 的核心模块演进，突出了 V2 的两大创新设计：倒残差结构与线性瓶颈。旨在解决 V1 在轻量化过程中遇到的特征表达能力下降问题。

一、模块结构对比解读

1. MobileNet V1 模块（上图）

•结构：输入 $\rightarrow 3 \times 3$ 深度卷积 $\rightarrow$ ReLU6 $\rightarrow 1 \times 1$ 逐点卷积 $\rightarrow$ ReLU6 $\rightarrow$ 输出

•设计逻辑：这是深度可分离卷积的直接、朴素实现。先按通道进行空间滤波，再融合通道信息。两个非线性激活函数试图增加模型的表达能力。

•存在的问题：

◦表征瓶颈：深度卷积在低维空间（通道数少）工作，其特征表达能力有限，紧接着的 ReLU 可能会进一步破坏这些已经稀疏的特征信息。

◦模块功能单一：仅作为一个轻量化的特征提取器，没有像 ResNet 那样的“恒等捷径”连接，不利于梯度和特征的流动。

2. MobileNet V2 模块（下图）

•结构：输入 $\rightarrow 1\times 1$ PW（升维，ReLU6） $\rightarrow 3\times 3$ DW（ReLU6） $\rightarrow 1\times 1$ PW（降维，Linear） $\rightarrow$ 输出

二、MobileNet V2 的核心创新详解

V2 模块的每一步都蕴含着特定的设计意图：

1. 第 1 步：1x1 逐点卷积（升维，带 ReLU6）

◦目的：扩展。将输入的低维特征（例如 24 维）映射到一个更高维的空间（例如 144 维）。这样做的理由是：在高维空间中，ReLU 激活函数的信息损失更小，非线性变换更加有效。这为后续的深度卷积提供了一个丰富的、高维的工作环境。

2. 第 2 步：3x3 深度卷积（带 ReLU6）

◦目的：滤波。在已经被升维的高维特征上进行轻量化的空间滤波。由于此时工作在富含信息的高维空间，深度卷积能够学习更有意义的空间特征，而 ReLU 激活造成的潜在信息损失也相对较低。

3. 第 3 步：1x1 逐点卷积（降维，带 Linear 激活）

◦目的：压缩 与 保护。这是 V2 最精妙的设计——“线性瓶颈”。

▪压缩：将特征映射回一个较低的维度（例如 24 维），以保持模型的紧凑和高效。

▪线性激活：为什么这里是 Linear 而不是 ReLU6？当在低维空间使用 ReLU 这样的非线性函数时，它会“裁剪”掉负值，可能导致大量信息被不可逆地破坏。由于我们刚把特征压缩回低维，这些特征非常珍贵。使用线性激活（即不做非线性变换）可以最大程度地保护从高维空间投影回来的信息完整性，避免表征瓶颈。

三、为何称为“倒残差”？

与经典的 ResNet 残差块（“压缩-处理-扩展”）相反，MobileNet V2 的模块是“扩展-处理-压缩”，因此得名“倒残差”。此外，V2 模块通常也包含一个短路连接，从模块的输入直接连接到模块的最终输出（在降维之后）。这个连接使得模块可以轻松地学习恒等映射，进一步稳定了训练，并让梯度能更有效地反向传播。

总结：从 V1 到 V2 的思维跃迁

特性	MobileNet V1	MobileNet V2
核心思想	解耦：简单地将卷积分解为两步。	重构与保护：在高维空间做非线性变换，在低维空间做线性保护。
结构顺序	滤波 → 融合	扩展 → 滤波 → 压缩 (倒残差)
激活函数	所有层后使用ReLU6	在瓶颈（低维）层使用线性激活
主要改进	实现了基础的计算效率。	1. 线性瓶颈保护了低维特征信息。 2. 倒残差结构在高维空间进行有效非线性变换。 3. 通常引入短路连接，提升训练稳定性。
效果	轻量化，但精度损失较大。	在同等甚至更低计算成本下，获得了显著的精度提升，更好地平衡了效率与性能。

结论：这张图完美地捕捉了轻量化网络设计中的一个关

键洞见——效率不仅来自于减少计算量，更来自于智能地安排计算和信息的流向。MobileNet V2 通过“先升维后降维”和“线性瓶颈”的协同设计，确保了在极致轻量化的前提下，网络仍能保有强大的特征学习和传递能力。这对于您关注的深山搜救等边缘计算场景至关重要，因为它意味着在无人机有限的算力上，能够运行一个既快又准的检测模型。

MnasNet:

创新点一：“多目标优化函数”

摒弃了以往仅追求高精度的思路，将“真实手机推理时间”直接作为优化目标之一，从而能自动搜索出在精度和速度之间取得最佳平衡的网络结构。下面这个表格概括了它的核心思想和实现方式。

核心要素	传统方法的问题	MnasNet的解决方案
优化目标	单一目标：最大化准确度	多目标：同时优化准确度和真实推理延迟 2 6
延迟度量	使用FLOPS（理论计算量）等间接代理指标，不准确 2 6	在真实手机上直接测量运行时间，作为延迟指标 1 6
核心公式	不适用	Reward = ACC(m) * [LAT(m) / T]^w • ACC(m): 模型精度 • LAT(m): 真实延迟 • T: 目标延迟 • w: 权重参数，控制惩罚力度 1 2

多目标函数的工作原理：

这个函数的核心在于，奖励值不仅取决于精度，还受到延迟与目标值之间关系的强烈影响。通过调整权重参数 w，可以实现不同的优化策略：

•硬约束策略：当设置 w 为 $\alpha=0, \beta=-1$ 时，函数对超过目标延迟 T 的模型进行严厉惩罚。这能确保搜索出的模型都满足延迟要求，但可能限制了搜索空间的多样性。

•软约束策略：当设置 w 为 $\alpha=\beta=-0.07$ 时，函数对延迟的响应是平滑渐变的。略超标的模型不会被直接淘汰，而是奖励值降低。这允许搜索算法探索更多样的速度-精度权衡，更可能发现意外优秀的模型结构。

为何强调“真实手机推理时间”？

这是 MnasNet 成功的关键。理论计算量（FLOPS）和在实际硬件上的运行速度之间存在巨大差异，这主要由内存访问开销和硬件并行优化程度不同导致。MnasNet 通过在真实移动设备上直接测量延迟，确保了搜索出的网络结构在真实场景中的高性能，使其结果远比依赖 FLOPS 等代理指标的方法更可靠。

如何在搜索中应用？

这个多目标奖励函数被集成到一个强化学习框架中：

控制器（一个 RNN） 根据当前策略“提议”一个网络结构。

训练器 训练该网络并得到其精度 $ACC(m)$ 。

手机 直接运行训练好的模型，测量其推理延迟 $LAT(m)$ 。

奖励计算 根据上述公式计算奖励值 $R(m)$ 。

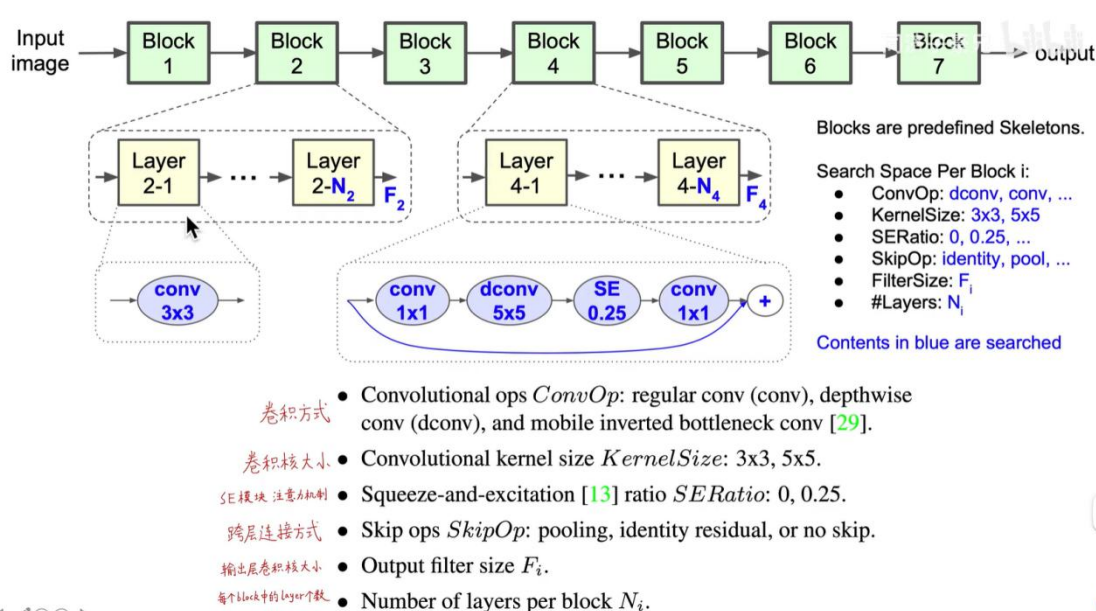
策略更新 奖励值反馈给控制器，用于更新其策略，使其倾向于提出更高奖励（即更快、更准）的结构。

总结：

MnasNet 的多目标优化函数是其灵魂所在。它将移动端部署的真实世界约束直接融入模型设计的最初阶段，通过平台感知的自动搜索，最终生成了在精度和速度上均超越当时主流手工设计模型（如 MobileNetV2）的网络结构，为移动端深度学习模型的开发提供了新范式。

创新点二：分层的 NAS 搜索空间

MnasNet 分层的 NAS 搜索空间这一核心设计。此设计完美回答了“如何在巨大的结构可能性中，高效地搜出一个最优网络”的问题。



一、核心思想：从“细胞堆叠”到“模块定制”

传统的 NAS（如 NASNet）像用同一种乐高积木块搭建整个模型。无论建筑底部（浅层、处理细节）还是顶部（深层、处理语义），都使用相同的“最优细胞”，这显然不符合卷积网络分层提取特征的本质。

MnasNet 则像一位总建筑师，他将建筑划分为地基、主体、屋顶等几个核心功能区块，并为每个区块独立设计最合适的内部结构。这就是分层的、模块化的搜索空间。

二、架构图详解：分层搜索的直观呈现

图片清晰地展示了这一过程的两个层级：

层级一：网络级规划（确定工厂车间）

- 流程：Input image→Block 1→Block 2→...→Block 7→Output

- 解读：MnasNet 将整个网络预先划分为 7 个连续的模块。这确定了搜索的“骨架”或“工厂的车间布局”。每个模块负责处理特定阶段和分辨率的特征。搜索将在每个模块的“配置清单”上进行，而非每一层。

层级二：模块级搜索（为每个车间定制机器）

这是搜索的核心。图片展开了 Block 2 和 Block 4 来示意。对于每一个模块(Block)，NAS 控制器需要从一份“配置清单”中为其选择一套统一的参数。一旦选定，该模块内的所有层（例如 Block 2 内的 Layer 2-1 到 Layer 2-N₂）都将采用此相同结构，即“块内同构”。

图中蓝色标注列出了这份关键的“可搜索配置清单”：

1. 卷积操作类型：是该用标准卷积、深度可分离卷积，还是更高效的逆瓶颈卷积？这决定了计算的基本单元。

2. 卷积核大小：用 3×3 还是 5×5 ？更大的核能捕获更广的上下文，但计算量也增加。

3. 是否添加 SE 模块：是否引入通道注意力机制来提升性能？比例是 0（不加）还是 0.25？

4. 跨层连接方式：是恒等连接、池化连接，还是无需连接？这影响了梯度和特征的流动。

5. 输出通道数：这个模块最终输出多少维特征？

6. 层重复次数：这个模块由几层相同的结构堆叠而成？（即图中的 N_i ）

三、设计优势：为何如此有效？

这种分层设计解决了传统 NAS 的两大痛点：

1. 搜索空间爆炸的诅咒：

◦如果为每一层独立选择结构，搜索空间大小为：选项数^{总层数}，这是天文数字。

◦MnasNet 通过“块内同构”，将搜索空间压缩为选项数^{模块数}。假设有 7 个模块，每层有 6 个选项，传统搜索空间是 6^L （ L 为总层数，可能 >50 ），而 MnasNet 则是 6^7 ，变得完全可操作。

2. 强制层多样性，实现精细权衡：

- 由于不同模块可以选择不同配置，网络天然获得了层多样性。

- 浅层模块（如 Block 1, 2）处理高分辨率图像，可能被自动搜索为“使用小卷积核、不加 SE 模块、通道数少”的轻量结构，以节省计算资源。

- 深层模块（如 Block 6, 7）处理丰富的语义信息，可能被搜索为“使用大卷积核、添加 SE 模块、通道数多、有残差连接”的高性能结构，以提升精度。

- 这种“因地制宜”的策略，让模型能在给定的延迟预算下，将“好钢用在刀刃上”，实现全局最优的速度-精度平衡。

四、总结：MnasNet 搜索空间的革命性

图片展示的，正是一个将自动化搜索的灵活性与先验的网络层次知识相结合的完美范例。它不再盲目地搜索每一层，而是以模块为单位，进行结构化、有层次的智能搜索。

最终结果：MnasNet 能够自动发现类似“在模型前期使用轻量操作节省时间，在模型后期使用复杂操作提升精度”这样的人类设计智慧，甚至能发现超越人类直觉的高效组合。这使其搜索出的模型在真实手机上的性能，全面超越了当时人工精心设计的 MobileNetV2 等模型。

这种分层搜索空间的思想，也成为了后续 EfficientNet 等系列模型的重要基础，极大地推动了移动端和边缘侧高效模型设计的发展。

ShuffleNet V1:

创新点一：分组 1×1 卷积

1. 要解决的问题：计算瓶颈

在轻量化模型（如 MobileNetV1）中，大量计算量（最高可达 95%）集中在 1×1 卷积（也称为逐点卷积）上。这是因为 1×1 卷积负责融合所有通道的信息，当通道数很大时，计算成本非常高。

2. 解决方案与原理

ShuffleNet 将分组卷积的思想应用到了 1×1 卷积上，称为分组 1×1 卷积。

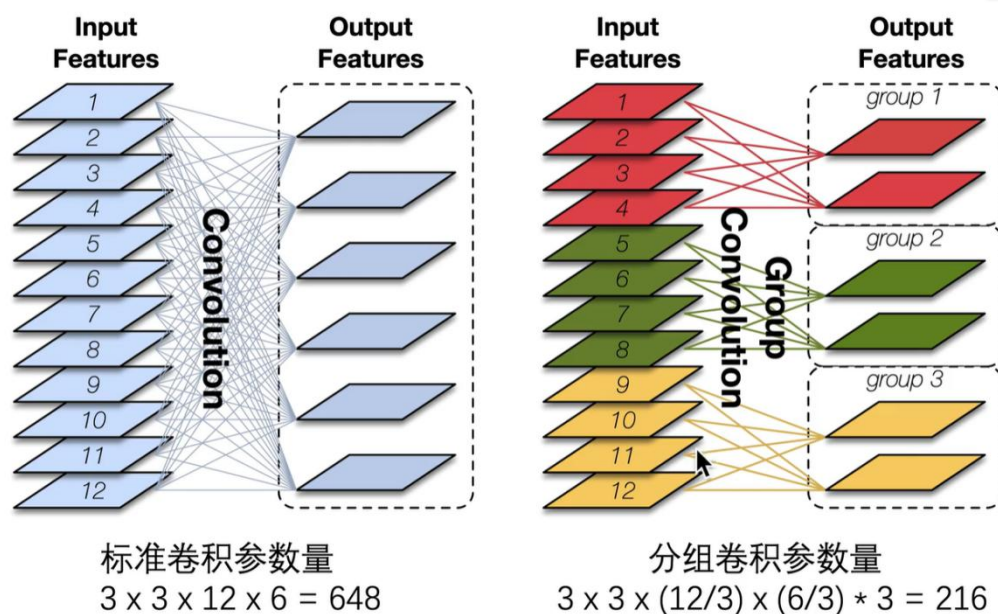
- 传统 1×1 卷积：每个卷积核都需要与输入的所有通道进行运算。

- 分组 1×1 卷积：先将输入的通道数分成 g 个组，每个组的卷积核只在与该组对应的输入通道上进行运算，最后将各组的输出拼接起来。

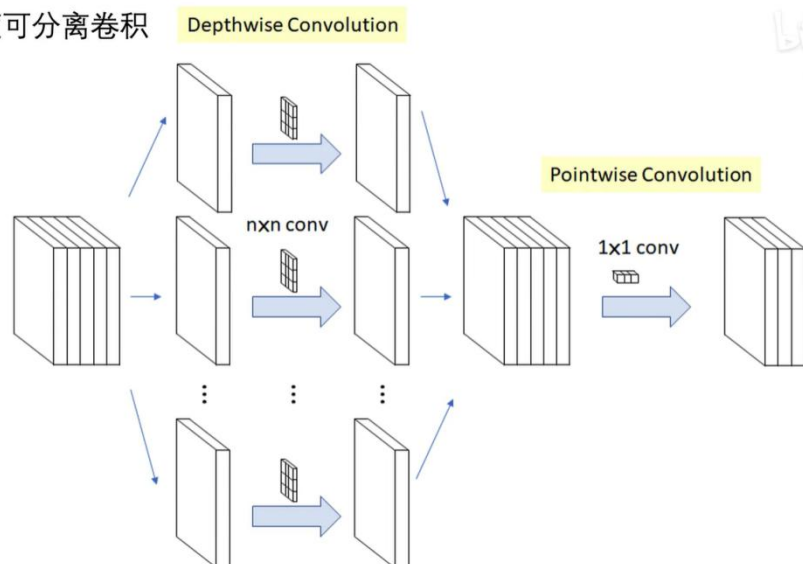
3. 带来的优势

这样做最直接的好处是大幅降低计算量和参数量。理论上的计算量和参数量都减少为传统 1×1 卷积的 $1/g$ 。节省出

来的计算预算，可以用来增加网络的宽度（通道数），让这个小模型能学习更丰富的特征，从而抵消因分组可能带来的表达能力下降。



MobileNet的深度可分离卷积
先Depthwise
再Pointwise



创新点二：通道重排

1. 新产生的问题：信息孤岛

然而，简单堆叠多个分组卷积层会带来一个严重副作用：“边界效应”。某一组的输出仅来自于上一层的同一组输入。

这导致组与组之间没有任何信息交流，形成了“信息孤岛”，严重削弱了模型的表征能力。

2. 解决方案与原理

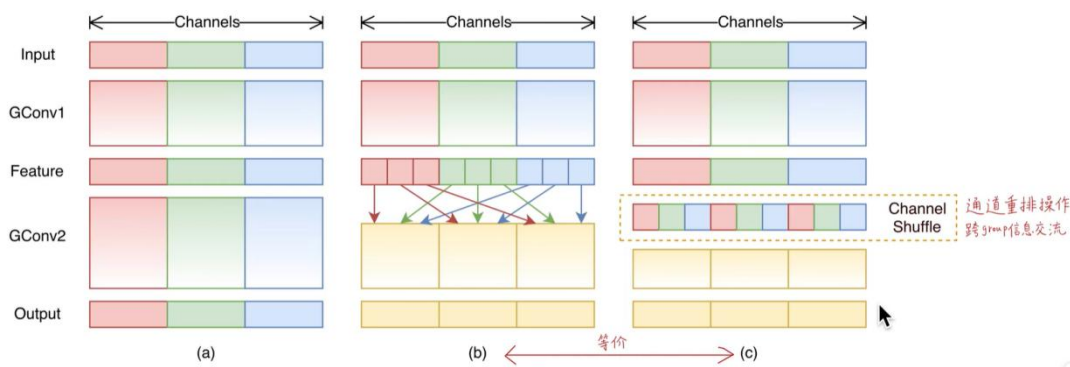
为了解决这个问题，ShuffleNet 提出了一个简单而优雅的“通道重排”操作。其步骤如下：

1. 重塑：将分组卷积输出的特征图（通道数为 $g \times n$ ）重塑为 (g, n) 的矩阵。
2. 转置：将这个矩阵进行转置，变为 (n, g) 。
3. 展平：最后将转置后的矩阵展平，作为下一层的输入。

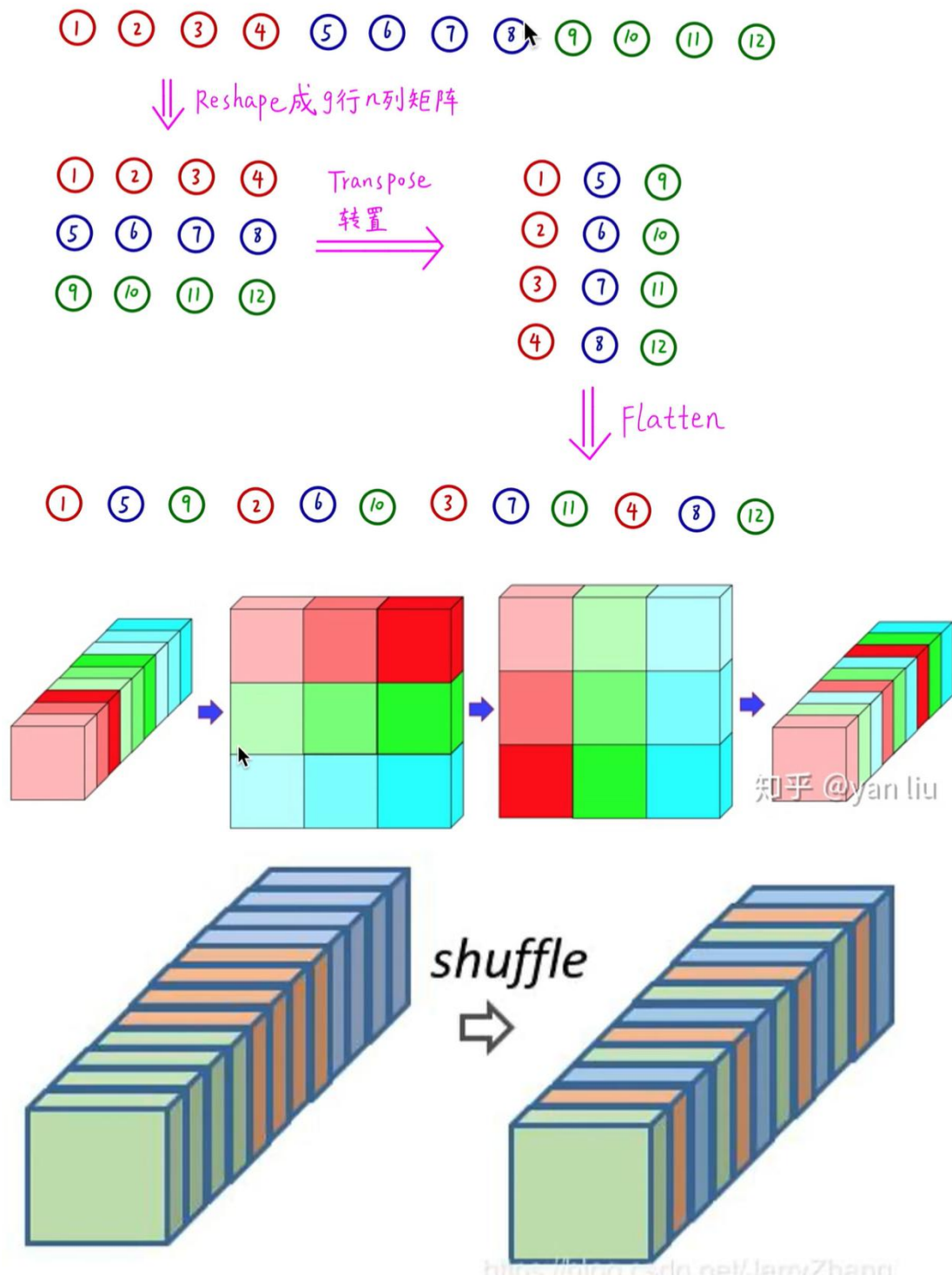
这个操作如同“洗牌”，它将不同组的子通道进行交叉重组，确保下一层分组卷积的每一个组都能接收到来自前一层的所有组的特征信息。

创新价值总结：1+1>2

ShuffleNet 的精妙之处在于，它通过“分组卷积”来极致地压缩计算成本，又通过“通道重排”这个低成本的操作来弥补分组带来的信息流通障碍。这两者结合，实现 1+1>2 的效果：



channel shuffle 通道重排过程



ShuffleNet V2:

四条轻量化网络设计原则:

准则一：输入输出通道数相同时，内存访问量（MAC）最小

- 内涵：在设计卷积层（尤其是轻量网络中常用的 1×1 卷积）时，应尽量使该层的输入通道数（ c_{in} ）与输出通道数（ c_{out} ）保持相等。

- 原因：卷积运算不仅涉及计算，还涉及大量的数据搬运（内存访问）。根据内存访问量的计算公式，当 $c_{in}=c_{out}$ 时，在固定计算预算下，所需的内存访问量达到理论最小值。这能最大程度缓解内存带宽瓶颈，提升实际速度。

- 在 ShuffleNet V2 中的应用：网络中的每个卷积块都严格遵守这一原则，块内卷积的输入输出通道数保持一致，形成了高效的数据流。

准则二：分组数过大的分组卷积会增加 MAC

- 内涵：虽然分组卷积（如 ShuffleNet V1 所用）能显著降低计算量（FLOPs），但过度分组（即组数 g 太大）会急剧增加内存访问量（MAC），反而可能拖慢速度。

- 原因：分组卷积会将连续的内存访问模式打散，增加内存访问的随机性和次数。当组数过多时，调度和数据收集的开销甚至会抵消计算节省带来的收益。

- 在 ShuffleNet V2 中的应用：ShuffleNet V2 完全摒弃了分组卷积，改用标准的密集连接 1×1 卷积。这使得它在获

得相近 FLOPs 的同时，具有更高的内存访问效率和实际运行速度。

准则三：碎片化操作对并行加速不友好

- 内涵：“碎片化操作”指的是网络中存在的多分支、多路径结构（例如 Inception 模块中的多个并行小卷积）。这类结构在理论上能丰富特征，但会降低硬件（尤其是 GPU、NPU）的并行计算效率。

- 原因：硬件喜欢大块、连续、统一的计算任务。碎片化结构引入了大量小型、不同类型的操作，迫使计算单元频繁切换和等待，无法充分利用其并行能力。

- 在 ShuffleNet V2 中的应用：ShuffleNet V2 采用了极其简洁的两分支结构。其核心是“通道分割”(Channel Split)操作：将输入特征图在通道维度上一分为二，一支路进行轻量变换，另一支路直接短路，最后合并。这种结构最大限度地减少了碎片化，提升了并行度。

准则四：逐元素操作（Element-wise）带来的内存和耗时不可忽略

- 内涵：逐元素操作（如 ReLU 激活函数、张量相加 Add、深度卷积 DWConv）的 FLOPs 虽然很低，但它们的内存访问量很大（需要读取和写入整个张量），且执行效率通常不高，累积起来会显著影响速度。

- 原因：这些操作的“计算/内存访问”比值很低，属于内存密集型操作。在带宽受限的移动芯片上，它们可能成为隐形的性能杀手。

- 在 ShuffleNet V2 中的应用：网络有意识地减少逐元素操作的数量。例如，只在关键位置使用激活函数，并尽可能将“相加”（Add）操作替换为“拼接”（Concat）操作，因为后者的实现通常更高效。

总结：从“计算量最优”到“速度最优”的范式转变

这四条原则共同指向一个核心思想：轻量化网络的设计目标，应从最小化理论计算量（FLOPs），转变为最大化在目标硬件上的实际运行速度。

ShuffleNet V2 正是基于这些从真实硬件 profiling 中得出的洞察，重新设计了网络单元，最终在相同精度下，相比 ShuffleNet V1 和其他轻量模型，在移动设备上实现了显著的实际速度提升。

对于您的深山搜救无人机项目而言，这四条原则是评估和选择骨干网络的黄金准则。一个符合这些原则的模型，更有可能在 Jetson 等边缘计算设备上发挥出预期的实时性能，确保搜救系统的效率。

知识蒸馏：

是一种将大型、复杂模型（教师模型）中蕴含的“知识”迁移到小型、轻量模型（学生模型）中的技术，旨在模型压

缩与加速的同时，力求保持其性能。其核心思想类似于人类社会经验丰富的教师指导学生高效学习的过程。

核心思想与工作原理

知识蒸馏的核心隐喻是师生教学。一个已经训练好的、性能强大但计算成本高的教师模型，将其学到的知识传授给一个结构更简单、参数更少的学生模型。其有效性基于一个关键洞察：教师模型不仅学会了如何正确分类（即识别“正确答案”），更在训练过程中学到了丰富的隐性知识，例如不同类别之间的相似性关系（比如猫和狗在某些特征上比猫和汽车更接近）。

为了实现知识迁移，知识蒸馏通常采用“软标签”作为知识的主要载体。

硬标签：传统的 one-hot 编码标签，如 $[1, 0, 0]$ 只包含“非对即错”的信息。

软标签：教师模型对各类别预测的概率分布，如 $[0.7, 0.25, 0.05]$ 包含了类别间的相似性等丰富信息。

hard targets

马 1

驴 0

汽车 0

soft targets

马 0.7

驴 0.25

汽车 0.05

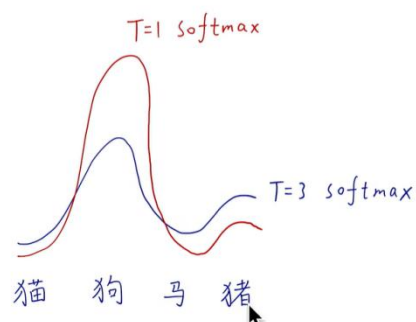
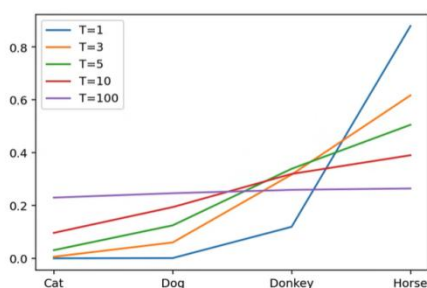
为了从教师模型的输出中提取更丰富的知识，引入了温度参数。具体操作是在 softmax 函数中加入温度参数 T ，以控制输出概率分布的“平滑度”：

$$q_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

T : 蒸馏温度

$T=1$ 时即为 softmax 同右式 $P(x_{ij})$

soft targets (z_i) $\xrightarrow{T \text{ 蒸馏}}$ q_i



- 高温 ($T>1$)：概率分布变得更平缓，负标签（非正确类别）所携带的差异信息被放大，便于学生模型学习更细微的知识。

- 低温 ($T=1$)：恢复为标准 softmax，输出分布更“尖锐”。

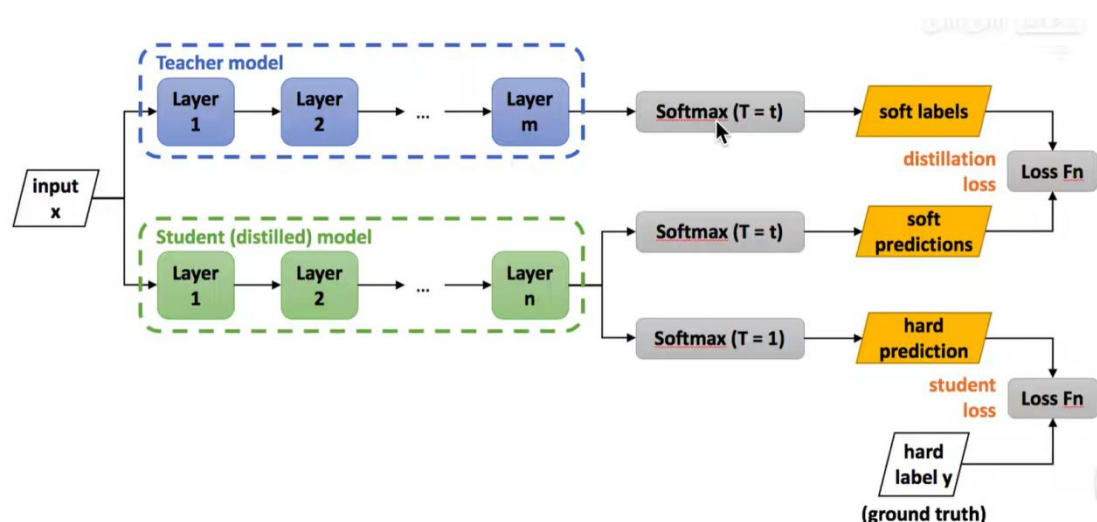
学生模型的目标之一，就是让自己的“软预测”（在高温 T 下计算）去逼近教师的“软目标”。

学生模型的训练目标由两部分损失函数共同指导：

1. 蒸馏损失：让学生模型的软标签（在高温度 T 下产生）去逼近教师模型的软标签。

2. 学生损失：让学生模型的硬标签预测（在温度 $T=1$ 下产生）去逼近真实的样本标签。

总损失是这两部分的加权和，从而使学生既能学到老师的“经验”，又不偏离正确答案。



知识的类型与蒸馏方式

除了上述基于模型最终输出的知识（基于响应的知识），知识蒸馏的技术范畴还扩展至其他层面：

- 基于特征的知识：让学生模型直接学习教师模型中间层的特征表示，这些特征通常包含更丰富的结构性信息，适用于计算机视觉任务。

- 基于关系的知识：让学生模型学习教师模型中不同数据样本或不同层特征之间的关系。

- 蒸馏的方式：

- 离线蒸馏：预先训练好教师模型，然后固定其参数，指导学生模型训练。这是最经典和常用的方式。

- 在线蒸馏：教师模型和学生模型同步训练，相互促进，适用于没有预训练强大教师模型的情况。

- 自蒸馏：模型自身充当教师和学生，例如同一网络的深层指导浅层，是一种有效的自我提升技术。

- 应用场景：

- 自然语言处理：将数百亿参数的大语言模型（如各类聊天机器人背后的大模型）压缩，使其能在手机或普通服务器上高效运行，实现快速文本生成、翻译等。

- 计算机视觉：将复杂的图像识别、目标检测模型蒸馏后部署于自动驾驶汽车、医疗影像设备等边缘设备，实现实时、低功耗的本地化智能。

- 语音识别：让智能手表、智能音箱等设备在离线状态下也能准确识别语音指令。

- 面临的挑战：

- 性能上限：学生模型通常难以完全复现教师模型在所有任务上的能力，尤其在需要复杂推理或创造性的任务上，性能可能会有差距。

- 知识残留：教师模型在一些罕见案例或细微模式上的判断逻辑可能在蒸馏过程中丢失。
- 偏见传递：若教师模型训练数据存在偏见，这种偏见会通过蒸馏过程传递给学生模型。
- 技术门槛：温度参数、损失函数权重等超参数需要精细调整，才能达到理想的蒸馏效果。

您这张手写笔记以非常生动的方式概括了机器学习的几大核心范式。左侧是规范术语，右侧是精妙的“人话”解读，两者结合，能帮助我们快速把握其精髓。以下是对图中八种学习方法的准确阐释及其在深山搜救等视觉任务中的潜在应用。

生成式对抗网络 GAN：

其核心思想是通过让两个神经网络相互博弈、共同进步，从而生成足以“以假乱真”的数据。

核心组件	角色比喻	核心目标	关键输入与输出
生成器 (Generator)	伪造者	学习真实数据的分布，生成尽可能逼真的假数据，以“欺骗”判别器。	输入：随机噪声向量 输出：伪造的数据（如图像）
判别器 (Discriminator)	鉴定专家	区分输入的数据是来自真实数据集还是生成器伪造的数据。	输入：真实数据或生成的数据 输出：一个0到1的概率值，代表“真”的可能性

核心原理：伪造与鉴定的博弈

可以把 GAN 的训练过程想象成一场伪造者（生成器）和鉴定专家（判别器）之间的较量。

1. 初始状态：伪造者技术拙劣，只能生成一些模糊或毫无意义的图案。鉴定专家虽然经验不足，但也能轻松识破这些粗劣的仿品。

2. 迭代提升：

①鉴定专家进修：利用真品和当前水平的仿品进行训练，鉴定专家的鉴伪能力得到提升。

②伪造者精进：伪造者根据鉴定专家提供的“反馈”（哪里被识破了），改进自己的伪造技术。

3. 理想终点：经过无数轮的这种对抗训练，两者都达到了极高的水平。伪造者能制作出连鉴定专家都难辨真伪的“完美仿品”，此时我们便得到了一个强大的生成模型。

从技术角度看，这被称为一个“极小极大博弈”问题。生成器试图最小化判别器的判断准确率，而判别器则试图最大化这个准确率，最终目标是达到一种平衡状态（纳什均衡）。

主要类型与技术演进

条件 GAN（CGAN）：给生成器和判别器都添加额外的条件信息（如标签或一段文字描述），从而实现可控生成。例如，可以指定生成“一只戴墨镜的猫”的图片。

深度卷积 GAN（DCGAN）：将卷积神经网络引入 GAN，极大地提升了生成图像的质量和训练的稳定性。

StyleGAN：能够生成极高分辨率（如 1024x1024）且细节极其逼真的人脸图像，并对如发型、姿势等属性进行精细控制。

CycleGAN：专注于图像到图像的转换，且无需配对数据。例如，可以将马的照片转换为斑马，或将夏日的风景照变为雪景。

广泛应用

GAN 的强大生成能力使其在众多领域大放异彩。

图像生成与编辑：这是 GAN 最广为人知的应用，包括创造虚构的人脸、艺术品、为黑白照片上色、提升图像分辨率（超分辨率）以及编辑照片中的面部属性（如年龄、表情）等。

数据增强：在医疗、工业检测等领域，标记数据可能很稀缺。GAN 可以生成逼真的合成数据，用于扩充训练集，从而提升其他诊断或检测模型的性能和鲁棒性。

跨领域创作：GAN 也被应用于文本生成（如对话、诗词）和音乐创作等领域，展现出其在非视觉领域的潜力。

挑战与局限

尽管强大，GAN 也面临一些挑战。

训练不稳定：训练过程可能难以收敛，有时甚至会彻底失败，需要精细的超参数调整。

模式崩溃：生成器可能只会产生几种单调的输出，无法捕捉真实数据完整的多样性。

GAN 在深山搜救中的核心应用场景：

GAN 在深山搜救中最大的价值在于解决“数据稀缺”和“场景覆盖不足”两大难题。

1. 生成多样化的训练数据：

挑战：深山环境复杂多变（如不同光照、天气、季节、植被密度），难以收集涵盖所有场景的真实数据。这会导致训练的检测模型泛化能力差，遇到未见过的情况时性能下降。

GAN 的解决方案：你可以利用 GAN 来生成大量、逼真的合成深山环境图像或红外小目标数据。例如，可以训练一个 GAN 模型，输入一张白天的山林照片，让其生成同一场景在浓雾、夜间、下雨或冬季等不同条件下的版本，并相应地生成在其中的“模拟待救人员”小目标。这能极大地扩充你的训练数据集，让模型见识到各种极端情况，从而更加鲁棒。

2. 进行数据增强

挑战：在真实数据量有限的情况下，模型容易过拟合，即只“记住”了有限的训练样本，而无法推广到新的搜救现场。

GAN 的解决方案：相比于传统的旋转、裁剪等简单数据增强手段，GAN 能进行语义层面的增强。它可以生成同一目标的不同形态、不同遮挡程度、不同尺度的变体。这对于你

申报书中关注的“微小目标”和“抗遮挡”问题尤其有用，能有效提升模型的检测精度和召回率。

潜在价值与优势

成本与安全性：在虚拟环境中生成数据，避免了在真实深山环境中进行大规模、高危数据采集的人力、物力和时间成本，且绝对安全。

定制化与可控性：可以针对特定的救援难点（如浓雾下的红外目标、密集树林中的遮挡目标）进行定向生成，快速制造大量有针对性的训练样本。

隐私保护：生成的数据是合成的，不涉及任何真实个人的隐私信息。

挑战与应对策略

尽管前景广阔，但将 GAN 应用于深山搜救也面临一些挑战，需要有清晰的应对思路：

挑战	说明与应对策略
训练难度与稳定性	GAN的训练过程以难以稳定著称，容易出现 模式崩溃 （生成样本多样性差）或训练发散 ^{1 6} 。 应对策略 ：优先选择训练更稳定的现代GAN变体（如Wasserstein GAN），并利用你在申报书中提到的 深度学习训练技巧 。
生成数据的真实性	如果生成的数据与真实深山环境 偏差过大 ，不仅无益，反而可能误导模型，导致性能下降。 应对策略 ：需要通过 人工评估 或 领域专家验证 来筛选高质量生成数据。一个关键的验证方法是：将“ 真实数据+生成数据 ”共同训练的模型，与仅用“ 真实数据 ”训练的模型进行性能对比。
计算资源需求	训练高质量的GAN模型需要 大量的计算资源 （高性能GPU）和时间 ⁶ 。 应对策略 ：可以从相对简单的环境（如低分辨率图像）开始实验，逐步迭代。

1. 对抗学习—“找对手互怼学习”

•核心：让两个神经网络（生成器与判别器）在对抗中共同进化，生成器试图生成以假乱真的数据（如图像），判别器则努力识别真伪。

- 深山搜救应用：可用于生成极端天气（浓雾、夜视）下的训练数据，或提升模型对模糊、遮挡目标的鲁棒性。

2. 自监督学习—“自己找标签学习”

- 核心：从无标签数据中自动构造监督信号进行预训练（例如，预测图像缺失部分、判断两张图像是否为先后顺序），学习通用特征表示。

- 深山搜救应用：在缺乏大量人工标注的深山红外/可见光图像上，预先学习关于地形、植被、物体的通用特征，为后续的小目标检测微调提供强大的特征提取器。

3. 弱/半监督学习—“标签：不够多，不够难，不够准”

- 核心：在标注成本高、标注质量不完美的现实条件下进行学习。“弱监督”指使用粗粒度、噪声标签；“半监督”指同时使用少量标注数据和大量无标注数据。

- 深山搜救应用：极为实用。搜救数据标注困难，可利用少量精确标注和大量易获取的无标签图像/视频进行训练，大幅降低对完全精确标注数据的依赖。

4. 知识蒸馏—“找老师教我学习”

- 核心：将大型、复杂、高性能的“教师模型”的知识，迁移到小型、轻量的“学生模型”中，在保持性能的同时提升推理速度。

- 深山搜救应用：将大型高性能检测模型的知识，蒸馏到轻量化网络中，以满足无人机等边缘设备的实时性和低功耗要求。

5. 多模态学习—“调动各类感官学习”

- 核心：融合多种类型的数据（如图像、红外、文本、声音）进行学习，使模型能获得更全面、互补的信息。

- 深山搜救应用：这是您申报书的核心。融合可见光图像（纹理细节）与红外热成像（温度信息），可实现在复杂光照、遮挡条件下，对失踪人员更稳定、更可靠的检测。

6. 迁移学习—“举一反三学习”

- 核心：将在源领域（如通用图像分类）上学习到的知识（模型参数、特征），迁移应用到目标领域（如深山小目标检测），以解决目标领域数据不足的问题。

- 深山搜救应用：标准做法和起点。使用在 ImageNet 等大型数据集上预训练的模型（如 VGG, ResNet）作为初始化，在深山搜救数据集上进行微调，能加速收敛并提升性能。

7. 集成学习—“三个臭皮匠学习”

- 核心：训练多个基学习器，并通过某种策略（如投票、平均）结合它们的预测结果，以期获得比单一模型更优越、更稳定的泛化性能。

- 深山搜救应用：可以集成多个基于不同架构或不同模式的检测模型，进行融合决策，降低漏报和误报，提升系统在复杂环境下的可靠性。

8. 联邦学习—“用别人的学习资料学习”

- 核心：在数据隐私保护的背景下，让多个参与方（如不同搜救队）在本地训练模型，只交换模型参数更新而非原始数据，共同建立一个全局的、更强大的模型。

- 深山搜救展望：未来可允许多个地方搜救部门协同训练一个更强大的全局模型，而无需共享其敏感的本地任务数据，保护行动隐私的同时提升整体智能水平。

一、为什么应该了解 YOLOv1? (学习的价值)

学习 YOLOv1 不是为了用它，而是为了理解现代目标检测的“根”。它的核心思想至今仍是 YOLO 系列的灵魂。

理解“端到端”与“统一检测”的革命性

在 YOLOv1 之前：主流方法（如 R-CNN 系列）是“分步走”——先找候选区，再对每个区域分类。速度慢，步骤复杂。

YOLOv1 的核心：“You Only Look Once”。它将检测框为一个单一的回归问题，用一个神经网络直接从输入图像预测出所有目标的类别和位置。这种“端到端统一检测”的范式是开创性的，也是后续所有版本乃至许多其他检测器的基石。

掌握最基础、最直观的检测框架

YOLOv1 将图像划分为 $S \times S$ 的网格，每个网格预测 B 个边界框和置信度。这个“网格预测”的思想非常直观。

理解了这种最朴素、最直接的框架，你才能深刻体会到后来版本中各种改进（如锚框、特征金字塔、解耦头）是为了解决什么问题。

看清技术演进的脉络

知道了 YOLOv1 的缺点（如：对密集小目标检测差、定位不够准），你就能秒懂后续版本的每个改进是为了什么：

为什么有 YOLOv2 (YOLO9000) 的锚框？→为了解决 v1

边界框预测不稳定的问题。

为什么有 YOLOv3 的特征金字塔？→为了解决 v1 对小目标检测能力弱的问题。

为什么后来版本用更复杂的损失函数？→为了解决 v1 简单平方和误差的不足。

这种“从问题到解决方案”的学习，比孤立地背诵每个版本的新特性要高效、深刻得多。

二、如何高效地学习 YOLOv1？（正确的方法）

你不需要去一行行读 YOLOv1 的原始代码（那是 C 写的，还很古老）。你应该采取“思想学习法”：

用 30 分钟看一篇经典解析文章或视频：快速搞懂它的网络结构、网格划分、损失函数设计这三大部分。

重点关注它的核心贡献：

将检测视为回归问题。

网格预测机制。

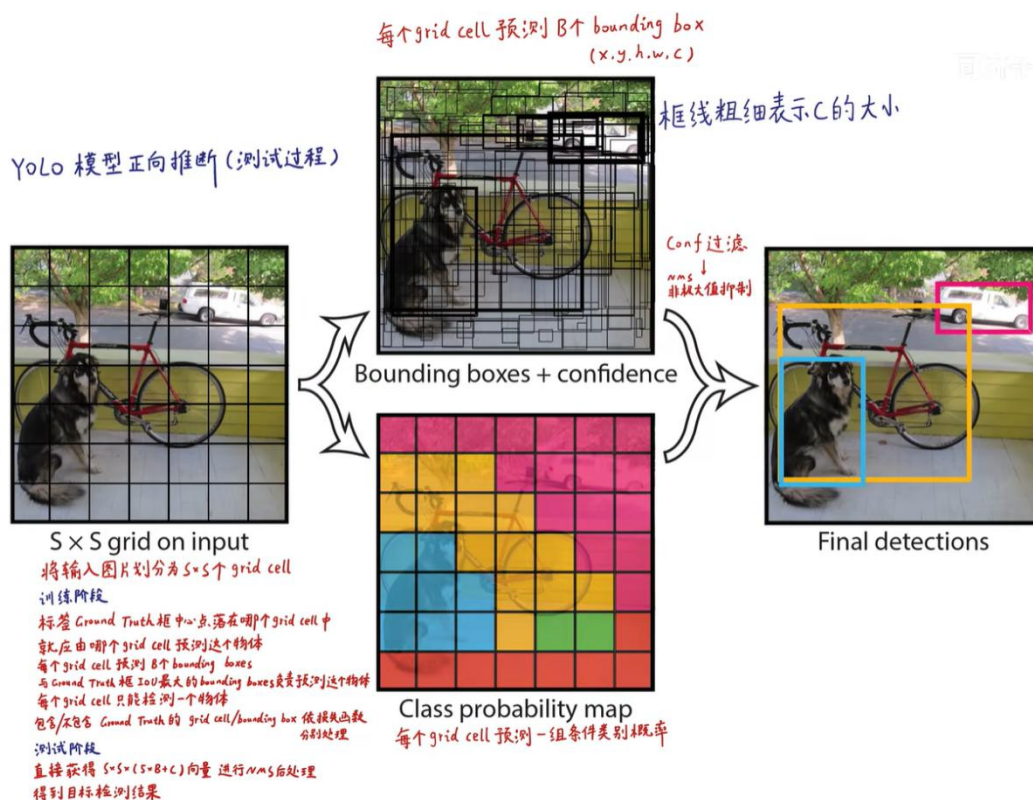
损失函数如何同时兼顾坐标、置信度和分类。

明确它的主要缺点（这更重要！）：

每个网格只能预测固定数量的目标，对密集和小目标不友好。

边界框预测不够灵活（没有锚框先验）。

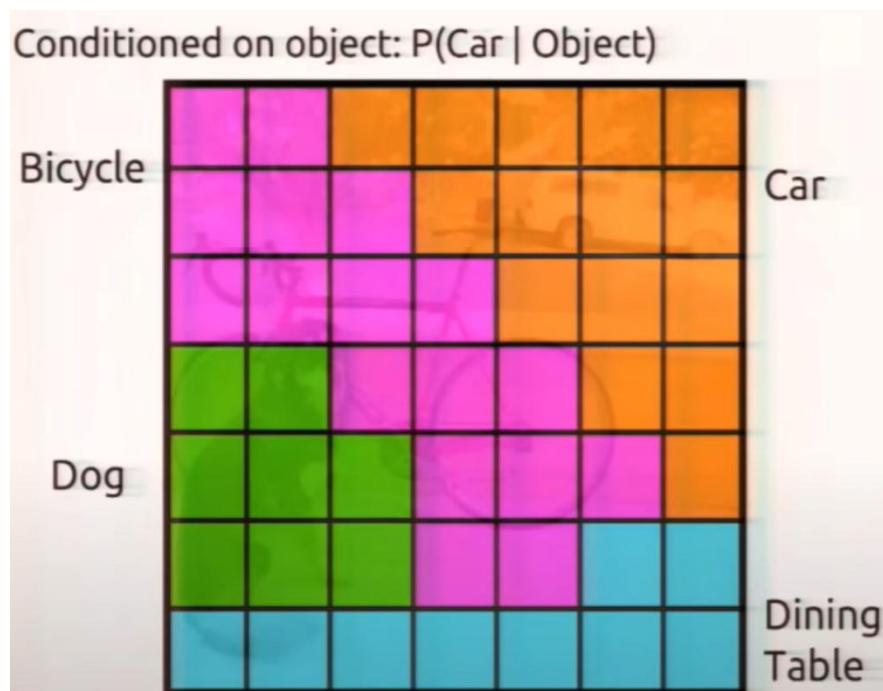
下采样倍数大，特征图分辨率低。



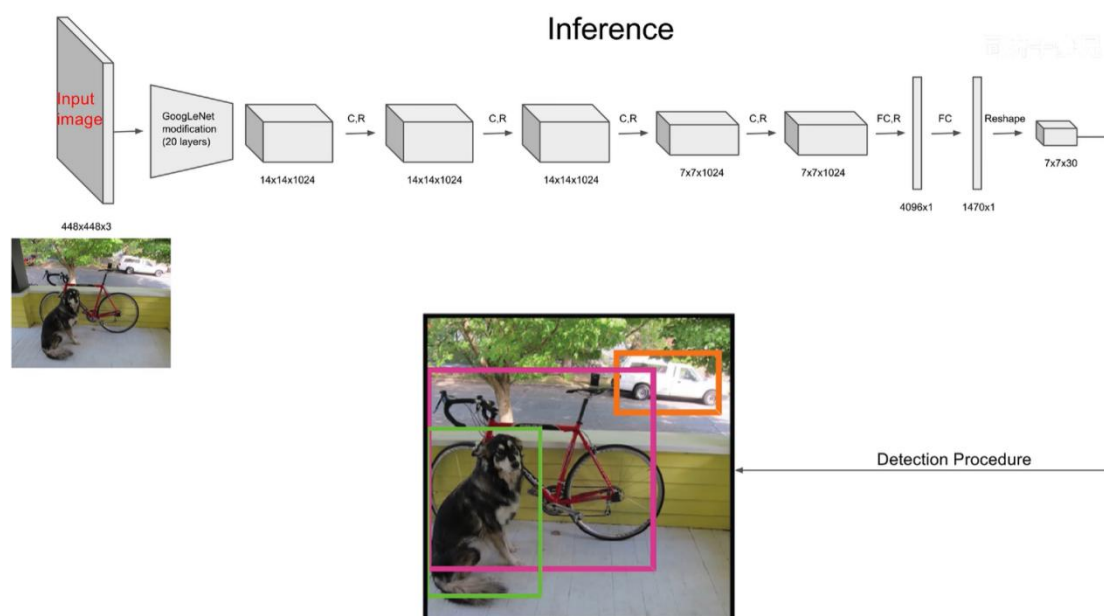
在 YOLOV1 中 $S=7$, $B=2$ 。

预测阶段：首先划分成 $S \times S$ 的网格 (grid cell)，每个网格又能预测出 B 个预测框 (bounding box)，这两个预测框的中心点都落在所在的网格里边，预测框的四个参数 (x, y, w, h) 即中心点的坐标和预测框的宽高，可由这四个参数来确定预测框的位置以及它是不是一个物体的置信度。图中间上面用黑框来表示置信度 (C) 的高低，框粗表示置信度高，框细表示置信度低。每一个网格 (grid cell) 还可以生成所有类别的条件概率 (也就是假设已经包含某个物体的情况下它是某一个类别的概率)，即图中间下面的彩色图，把每一个预测框的置信度乘以类别的条件概率就能够获得每

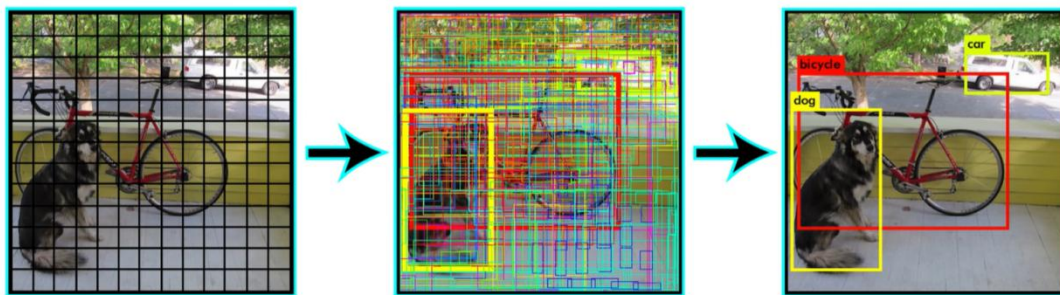
一个预测框的各类别的概率。结合这两个信息：预测框的信息与网格的类别信息，就可以获得最后的预测结果。



把预测出来的 98 个预测框进行筛选，最后只保留一个作为最终的预测结果，这个过程称作 NMS 非极大值抑制。



下图中间部分：不同粗细的框表示置信度不同；不同颜色的框表示类别不同。



YOLO V1 损失函数

每一项都是平方和误差 将目标检测问题当作回归问题
批注: 同济子豪兄

loss function:

$$\begin{aligned}
 \text{loss function:} &= \lambda_{\text{coord}} \sum_{i=0}^S \sum_{j=0}^{S^2} \mathbb{I}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] \\
 &+ \lambda_{\text{coord}} \sum_{i=0}^S \sum_{j=0}^{S^2} \mathbb{I}_{ij}^{\text{obj}} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right] \\
 &+ \sum_{i=0}^S \sum_{j=0}^{S^2} \mathbb{I}_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 \\
 &+ \lambda_{\text{noobj}} \sum_{i=0}^S \sum_{j=0}^{S^2} \mathbb{I}_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2 \\
 &+ \sum_{i=0}^S \mathbb{I}_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2 \quad (3)
 \end{aligned}$$

负责检测物体的bbox 中心点定位误差
 负责检测物体的bbox 宽高定位误差
 求根号能使+对误差更敏感
 负责检测物体的bbox Confidence 误差
 不负责检测物体的bbox Confidence 误差
 负责检测物体的grid cell 分类误差

第i个grid cell 是否包含物体
 也即是否有ground truth 矩形的中心点落在该grid cell 中
 若有则为1 否则为0
 第i个grid cell 的
 第j个bounding box
 若负责预测物体则为1
 否则为0
 第i个grid cell 的
 第j个bounding box
 若不负责预测物体则为1
 否则为0

坐标回归误差
 预测值 标签值
 计算这个bbox 与 ground truth 的IoU
 从模型5个输出结果中 (0-1) 维向量中找到这个IoU的Confidence Score
 类别预测误差

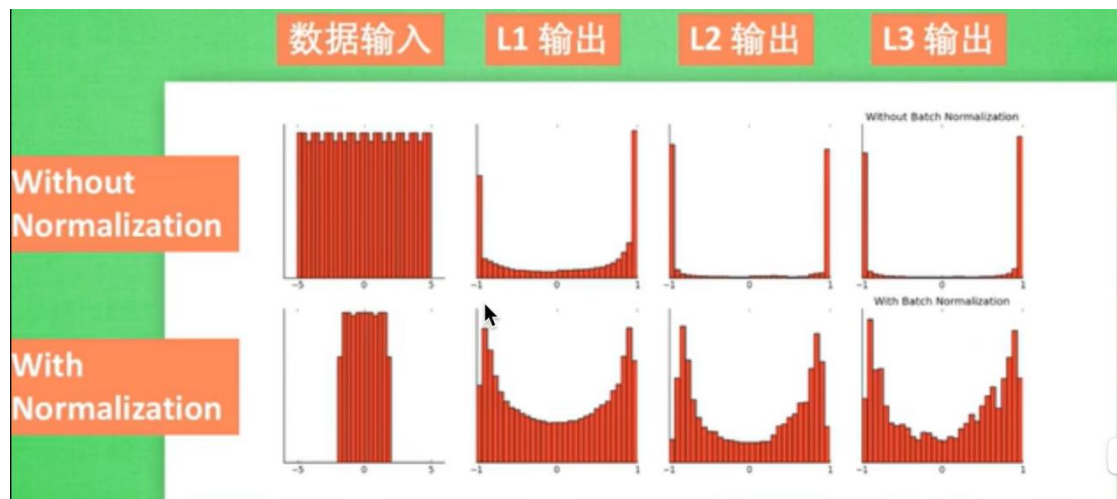
YOLOV1 缺点：准确性低；定位性能较差；recall 低(就是把全部物体全部类别检测出的能力差)；检测小目标和密集目标的能力差。

YOLOV2:

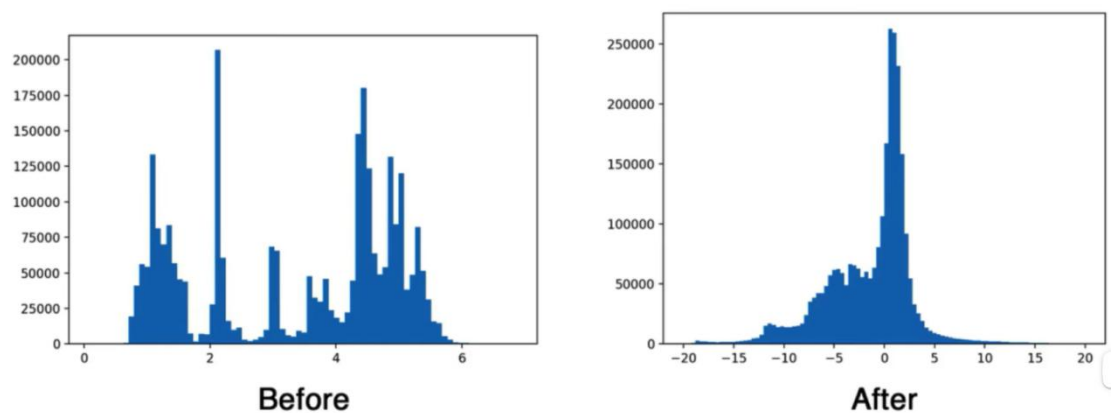
1. Batch Normalization:

BN 层就是把神经元的输出减去均值除以标准差，将其变

为一个以 0 为中心以 0 为均值，标准差为 1 的一个分布。之所以这样操作是因为很多激活函数在 0 附近是非饱和区，如果输入太大或太小，就会陷入这个激活函数的饱和区，就意味着梯度消失，难以训练。



- Batch normalization – increase of 4% mAP



训练阶段

例如batch size为32，某层的某个神经元会输出32个响应值。

对这32个响应求均值、标准差，再做归一化。

把归一化后的响应值乘 γ 再加上 β 。

每个神经元都训练一组 γ 、 β 。

测试阶段

测试阶段均值、方差、 γ 、 β 都用训练阶段全局求出的。

Batch Normalization相当于做线性变换。

$$\mu_{test} = \mathbb{E}(\mu_{batch})$$

$$\sigma_{test}^2 = \frac{m}{m-1} \mathbb{E}(\sigma_{batch}^2)$$

$$BN(X_{test}) = \gamma \cdot \frac{X_{test} - \mu_{test}}{\sqrt{\sigma_{test}^2 + \epsilon}} + \beta$$

BN 层一般在线性函数的后面，激活函数的前面。

BN 层可以加快神经网络的收敛；改善梯度（远离饱和区）；可以使用大的学习率来训练网络；其对初始化不敏感；起到了一个正则化的作用，可以防止过拟合，甚至可以取代 Dropout 层。

训练和测试时的 BN 层不一样。

BN 和 Dropout 可以一起使用，但在很多现代深度网络中，它们一起使用可能效果不佳，甚至有害，因此经常被建议避免同时使用。

一、核心思想回顾：它们在做什么？

- 批量归一化（BN）：它的核心目标是 “稳定和加速训练”。在每一层的输入处，BN 会强制将该批次（Batch）的数据分布调整为均值为 0、方差为 1 的标准正态分布。这极大地缓解了内部协变量偏移问题，使得网络可以使用更大的学习率，收敛更快、更稳定。BN 在训练和推理时的行为是不

同的：训练时使用当前批次的统计量（均值、方差），并更新全局的移动平均值；推理时则使用训练阶段累积的固定全局统计量。

•Dropout：它的核心目标是“防止过拟合”。在训练时，它以概率 p 随机将神经元激活置零。这可以看作是在训练一个“神经网络的集合”，并迫使每个神经元不过度依赖于少数其他神经元，从而增强模型的泛化能力。Dropout 在训练和推理时的行为也是不同的：训练时进行随机丢弃；推理时则将所有激活乘以 $1-p$ （缩放）或使用其他近似方法，以“模拟”多个网络的平均效果。

二、为什么一起使用可能出问题？（冲突根源）

BN 和 Dropout 在训练和测试时的行为模式差异是导致问题的核心。

1. 训练与推理的“行为不一致”被放大

◦当我们单独使用 Dropout 时，训练时的随机失活和推理时的确定性缩放之间已经存在一个“近似”。这个近似在大多数情况下是有效的。

◦当我们把 Dropout 放在 BN 之前（常见做法：Conv $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Dropout），问题就来了：在训练时，Dropout 会随机改变输入到下一层 BN 的统计分布。例如，某一层的输出在某一批数据中，因为 Dropout 丢弃了不同的神经元，其均值和方差会产生剧烈波动。

◦而 BN 在训练时，会基于这个被 Dropout 随机扰动过的、不稳定的批次来计算均值和方差，并更新其全局移动平均统计量。这导致 BN 在训练阶段学到的全局统计量，是基于一个“噪声很大”的分布估计的。

◦到了推理阶段，Dropout 关闭，激活模式变得确定且平滑，此时 BN 使用的却是基于“噪声训练分布”学到的固定统计量。这导致推理时 BN 的归一化操作所基于的统计量与数据真实的统计量出现系统性偏差，破坏了 BN 稳定分布的初衷。

2. 方差的偏移 (Variance Shift)

◦这是一个更理论化的解释。Dropout 操作本身会改变神经元输出的方差。当 Dropout 层后面紧跟着 BN 层时，BN 的缩放参数 (γ) 和偏置参数 (β) 会尝试去适应和补偿由 Dropout 引起的方差变化。

◦然而，在推理时 Dropout 被关闭，方差又回到了原始状态，但 BN 的参数已经为适应“有 Dropout 的状态”而调整过了。这种训练与推理状态下方差不匹配的现象，被一些研究称为“方差偏移”，会损害模型的最终性能。

三、现代最佳实践与建议

基于以上冲突，学术界和工业界形成了以下共识：

1. 在具有 BN 的现代深度网络中(如 ResNet, DenseNet, EfficientNet 及其变体)，通常可以省略 Dropout。

◦原因：BN 本身具有一定的正则化效果（因为它在训练时使用小批次的统计量，引入了噪声）。更重要的是，现代网络通过使用大量数据、权重衰减、数据增强、标签平滑以及更宽/更深的架构（本身就有很好的泛化能力）来防止过拟合。Dropout 带来的收益变得有限，而其与 BN 的副作用可能超过其正则化好处。

2. 如果必须使用 Dropout，请将其置于网络最后（例如分类器之前），而不是与 BN 层紧邻。

◦一个常见的、相对安全的模式是：

主干特征提取网络（大量使用 BN） → Global Pooling → Dropout → Final Fully-Connected Layer

这样，Dropout 作用于最终的全连接层，避免了与中间层 BN 的直接交互。

3. 选择其他与 BN 兼容性更好的正则化方法：

◦数据增强：始终是最有效、无副作用的正则化手段。

◦权重衰减：在优化器中直接对权重进行 L2 惩罚。

◦标签平滑：防止模型对训练标签过度自信。

◦Stochastic Depth 或 DropPath：在一些架构（如 Vision Transformer，特别是 Swin Transformer 中常用的 Stochastic Depth）中，随机丢弃整个网络块，而不是单个神经元，这种“结构化丢弃”与 BN 的兼容性更好。

◦LayerNorm / GroupNorm + Dropout：在 RNN/Transformer 等序列模型中，常用 LayerNorm 代替 BN。因为 LN 不依赖批次统计量，所以与 Dropout 的兼容性非常好（这也是为什么在 Transformer 中 Dropout 被广泛使用的原因）。

总结		
特性	批量归一化	Dropout
主要目的	稳定训练，加速收敛	防止过拟合，提升泛化
工作原理	规范每层输入的分布	随机失活神经元
与对方的冲突	依赖稳定的批次统计量	引入随机扰动，破坏统计量稳定性
现代实践	CNN骨干网络的核心组件	在含BN的CNN中谨慎使用或避免；在FC层后或Transformer中常用

现代实践 CNN 骨干网络的核心组件 在含 BN 的 CNN 中谨慎使用或避免；在 FC 层后或 Transformer 中常用

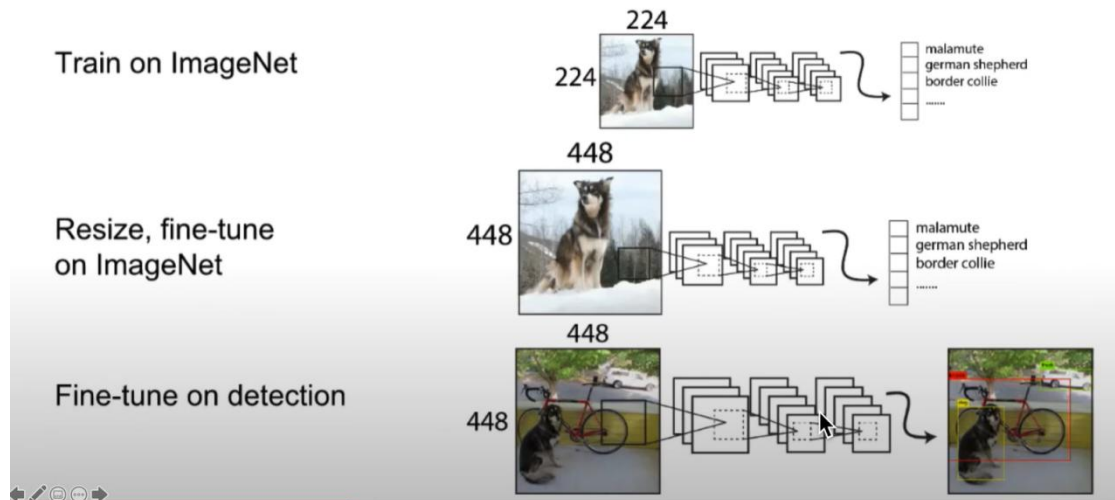
结论：

在构建以卷积神经网络（CNN）为主、大量使用 BN 的视觉模型时，默认情况下不应该在卷积层或 BN 层之后插入 Dropout。BN 的强大正则化能力和现代训练技术通常已足够防止过拟合。

如果你的模型仍然过拟合，首先应该考虑增加数据增强、使用权重衰减、或简化模型，而不是强行加入 Dropout。只有在非常特定的场景下（如网络最后的全连接层），经过仔细的消融实验验证后，才考虑将两者结合使用。

2.High Resolution Classifier：（高分辨率的分类器）

Fine-tune 448x448 Classifier: +3.5% mAP



这张图非常清晰地解释了 YOLOv2 中一项关键且有效的改进：通过高分辨率分类器微调来提升检测精度。它形象地展示了从“分类网络”到“检测网络”的三步渐进式训练策略，正是这个策略贡献了+3.5% mAP 的性能提升。

第一步：在标准分辨率（224x224）上训练一个基础的分类网络

- 目标：让网络学会识别物体（“这是什么？”）。
- 做法：使用大规模分类数据集（如 ImageNet），以 224x224 像素的标准输入分辨率，训练一个分类模型（例如 DarkNet-19）。

•结果：网络掌握了强大的特征提取能力，能区分成千上万种物体类别（如图中的各种犬种）。这为后续的检测任务打下了坚实的基础。

第二步：将分类器微调到高分辨率（448x448）

- 目标：让网络适应更高的输入分辨率，从而学习更精

细的图像特征。

- 做法：将上一步训练好的分类网络，其输入分辨率从 224×224 调整为 448×448 。然后，继续在 ImageNet 数据集上对这个分辨率更大的分类任务进行短期微调（大约 10 个 epoch）。

- 关键原因：

- 检测任务需要精确的边界框定位，而低分辨率训练会丢失很多细节信息。

- 直接将一个在 224×224 上训练好的分类网络，切换到 448×448 去做检测，网络会因输入分布的巨大变化而感到“不适应”。

- 因此，先在分类任务上完成这个“分辨率切换”的适应过程，能让网络平滑过渡，为下一步的检测训练做好充分准备。

- 图示：图中中间的狗图像变得更大、更清晰，正是体现了分辨率提升后，网络能“看到”和“理解”更丰富的细节。

第三步：在高分辨率（ 448×448 ）上微调检测网络

- 目标：将网络从“识别物体”的能力，转变为“定位并识别物体”的能力。

- 做法：将第二步得到的高分辨率分类网络作为检测网络的主干（Backbone），移除其最后的分类层，替换为检测

专用的检测头（用于预测边界框和类别）。

- 训练：在目标检测数据集（如 PASCAL VOC, COCO）上，以 448x448 的输入分辨率，对整个网络进行端到端的微调。

- 结果：此时，网络不仅利用了从 ImageNet 学来的强大特征，还适应了高分辨率输入，从而能更精准地预测物体的位置（边界框）和类别。图中最下方出现了带有“狗”和“自行车”检测框的街道场景，正是最终检测网络的输出。

总结与核心洞见

这张图揭示了 YOLOv2（以及许多现代检测器）一个至关重要的设计哲学：“分阶段训练” 和 “高分辨率迁移”。

1. 为什么能提升 3.5% mAP？

- 直接跳跃（从 224 分类 → 448 检测）会让网络同时面对分辨率变化和任务变化（分类变检测）两大挑战，导致训练不稳定，难以收敛到最优。

- YOLOv2 聪明地插入了一个 “448 分辨率分类微调” 的中间步骤。这让网络先专注解决分辨率适应问题，在熟悉的分类任务上稳定下来，提取更精细的特征。

- 随后，在这个已经适应了高分辨率的、特征更强的网络上进行检测微调，起点更高，自然能获得显著的精度提升。

2. 与 YOLOv1 的对比：

- YOLOv1 是直接在 448x448 分辨率下，从随机初始

化的权重开始训练检测网络。这种方式训练难度大，且没有充分利用在分类任务上预训练的优势。

◦YOLOv2 的改进，使得检测网络能够更充分、更平稳地继承大型分类数据集（ImageNet）的知识，这是其性能超越 v1 的重要原因之一。

总而言之，这张图告诉我们： YOLOv2 通过“先在低分辨率训练分类 -> 然后在高分辨率微调分类 -> 最后在高分辨率微调检测”这条精妙的路径，成功地将分类网络的强大特征提取能力，平滑、高效地迁移到了高分辨率的检测任务中，从而用相对简单的技巧换取了显著的性能提升（+3.5% mAP）。这是一个将迁移学习思想发挥到极致的经典范例。

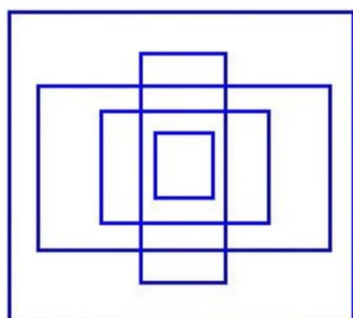
3. Anchor:

YOLOV1 没有使用 Anchor，而是把整张图化成了 7×7 的网格，每个网络预测出 2 个 bounding box，这 2 个 bounding box 哪个与 ground truth 的 IOU 大，谁就去负责拟合 ground truth。

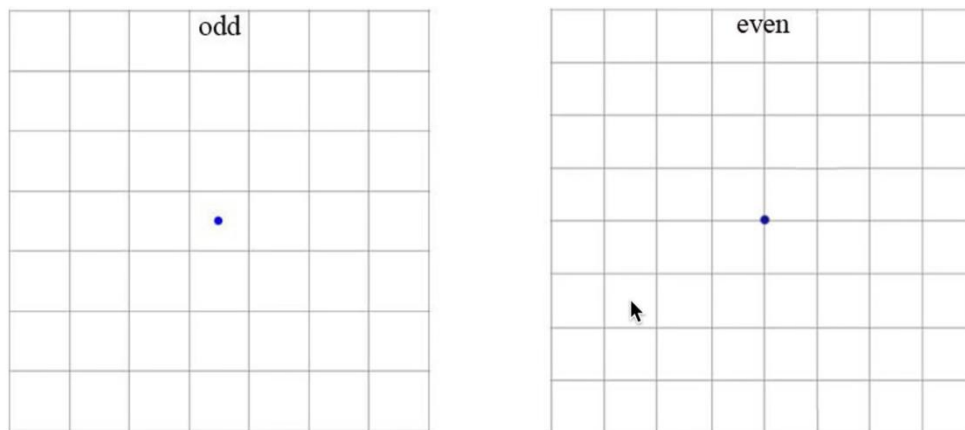
而在 YOLOV2 中，把一张图片划成了 13×13 个 grid cell，每一个 grid cell 预测 5 个 Anchor，也就是说事先指定了 5 种长宽、大小、尺度不同的先验框，每一个 Anchor 都对应一个预测框，而这个预测框只需要取预测输出它相对于它所在的 Anchor 的偏移量即可。



比如这个小女孩，人工标注框的中心点落在了哪个 grid cell 里面，就应该由哪个 grid cell 产生的这 5 个 Anchor 中与 ground truth 的 IOU 最大的 Anchor 去负责预测，而这个预测框只需要预测出它相较于自己 Anchor 的偏移量即可。



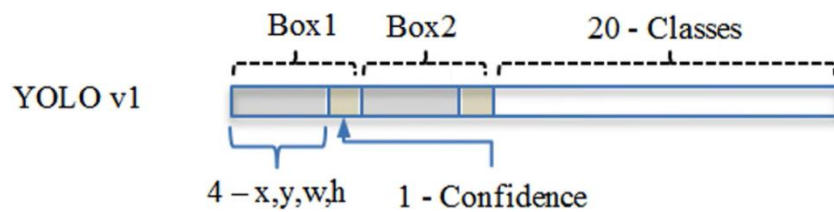
论文提到，模型输出的 feature map 弄成了奇数长宽，因为其有一个中心的 grid cell 去预测，而不是偶数的 featur map 也就是 4 个 grid cell 去预测。



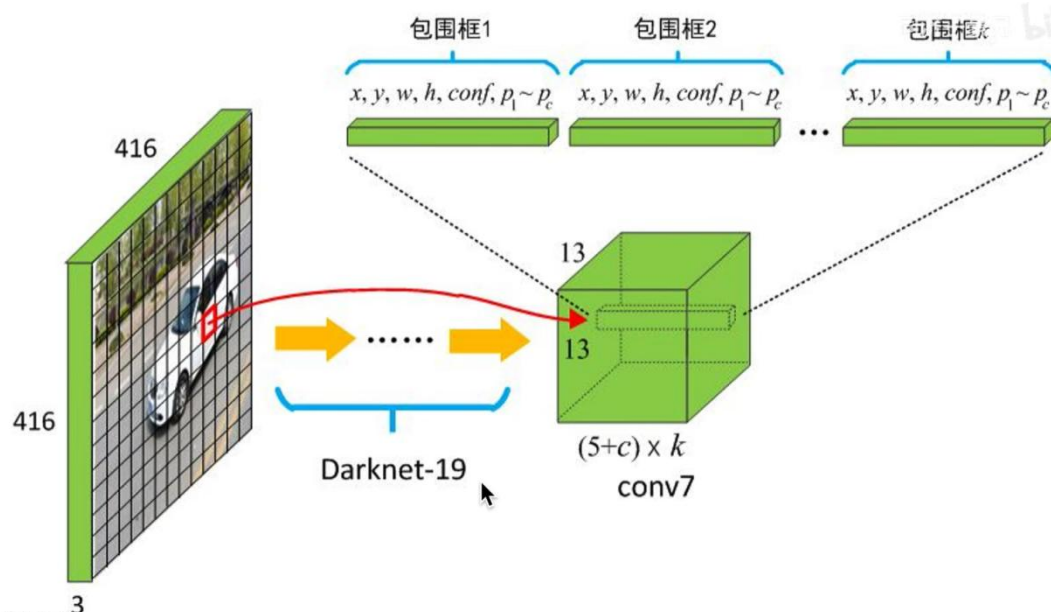
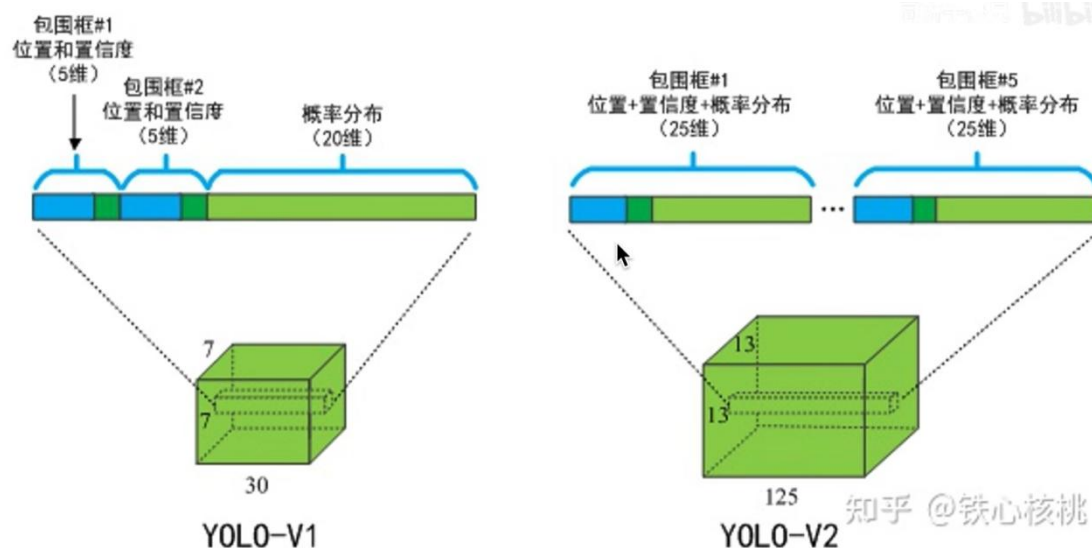
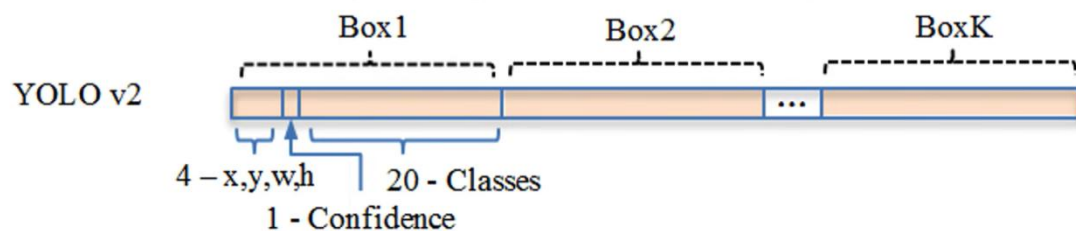
模型输出结构也进行了改变。YOLOV1 里没有用 Anchor，划分成 7×7 个 grid cell，每个 grid cell 输出 2 个 bounding box，每一个 bounding box 有 4 个位置参数 (x, y, w, h) 和 1 个置信度。grid cell 还预测出 20 个类别的条件类别概率。所以就是 $5 \times b + c = 5 \times 2 + 20 = 30$ ，YOLOV1 中每一个 grid cell 产生 30 个数，构成 30 维的向量。

在 YOLOV2 中，把整个图片划分成 13×13 的 grid cell，每一个 grid cell 产生 5 个 Anchor，每一个 Anchor 除了 4 个位置参数和 1 个置信度外，还有 20 个类别的条件类别概率。所以每一个 Anchor 都会产生 $4 + 1 + 20 = 25$ 个数，5 个 Anchor 构成 125 个数，所以一个 grid cell 产生 125 个数，构成 125 维向量 (5×25 维的矩阵)。在三维空间也就是 $13 \times 13 \times 25$ 的张量。

最终目标检测的结果就是从这个张量中提取出来的，所以其是单阶段方法，是端到端的，输入图片然后输出张量。



<http://blog.csdn.net/linolzhang>



图中的 conf 为置信度，p1-pc 为类别概率，c 是类别个数 20，k 是 Anchor 数量 5。

可以用聚类的方法选 Anchor。

Box Generation	#	Avg IOU
Cluster SSE 欧氏距离	5	58.7
Cluster IOU	5	61.0
手动选 Anchor Faster R CNN Anchor Boxes [15]	9	60.9
Cluster IOU	9	67.2

中心坐标预测 (b_x , b_y):

$$b_x = \sigma(t_x) + c_x$$

$$b_y = \sigma(t_y) + c_y$$

解读:

c_x , c_y 是当前网格左上角的坐标（以网格为单位，例如第 2 行第 3 列的网格，其 $c_x, c_y=1$ ）。

t_x , t_y 是网络直接预测的偏移量。

σ 是 sigmoid 函数，它将 t_x , t_y 压缩到 (0, 1) 之间。

• 这意味着，预测框的中心点被限制在了当前网格内部 (c_x 到 c_{x+1} 之间)。这是一个非常关键的设计，它让每个网格专注于预测中心点在自己区域内的物体，极大地稳定了训练过程，并加速了收敛。

宽度和高度预测 (b_w , b_h):

$$b_w = p_w * e^{\hat{t}_w}$$

$$b_h = p_h * e^{\hat{t}_h}$$

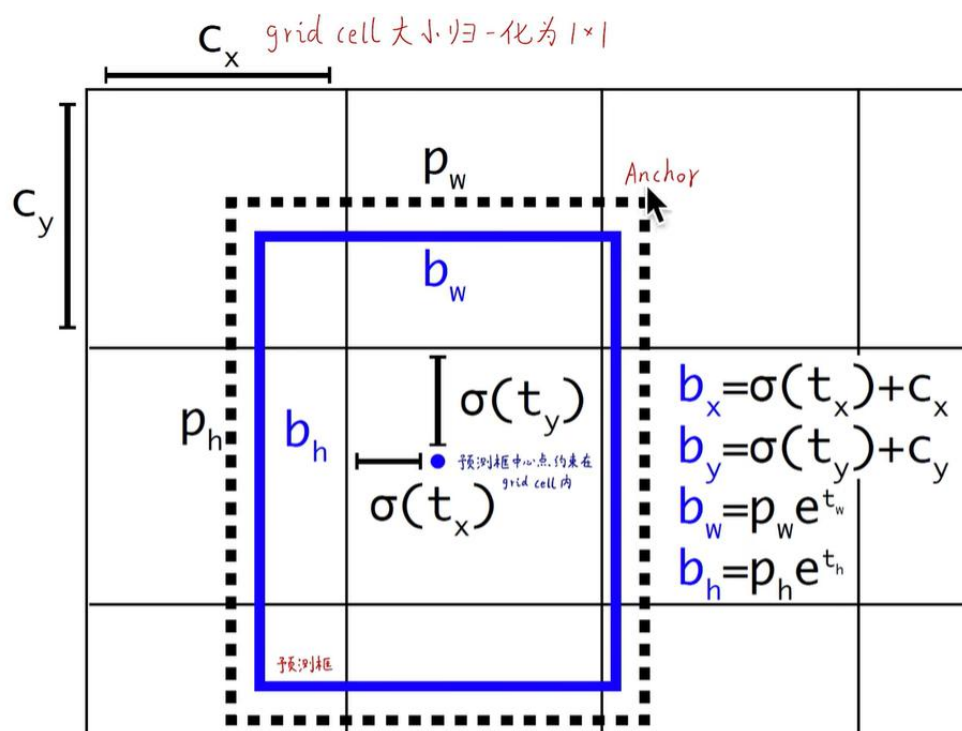
解读：

p_w, p_h 是预设的锚框（Anchor）的宽和高。

t_w, t_h 是网络预测的缩放系数。

$e^{\hat{t}_w}$ 和 $e^{\hat{t}_h}$ 保证了缩放因子为正数。

这意味着，网络不是直接预测宽高，而是预测相对于锚框的“宽高应该缩放多少”。由于锚框的形状是通过训练集聚类分析得到的（例如，数据集中有很多瘦高的行人框和扁平的汽车框），网络只需要在这个“经验形状”的基础上进行微调，这比从零开始学习所有形状要容易得多。



预测输出 (t_x, t_y, t_w, t_h)

在图中，这四项 (t_x, t_y, t_w, t_h) 就是网络在每个网格、对于每个锚框需要直接输出的值。它们和置信度以及类别概

率一起，构成了网络最终的预测向量。

总结：YOLOv2 预测机制的优势

通过这张图，我们可以总结出 YOLOv2 边界框预测的三大优势：

解耦了位置与形状预测：中心点预测通过 sigmoid 函数约束在网格内，宽高预测通过 exp 函数在锚框基础上缩放，两者各司其职，学习目标更明确。

引入了先验知识：使用聚类得到的锚框作为形状先验，让网络学习“调整”而非“创造”，降低了学习难度，特别是对于不同长宽比的目标。

提升了召回率与稳定性：每个网格可以预测多个（例如 5 个）不同形状的锚框，这大大增加了模型覆盖不同大小和比例目标的能力，同时稳定的预测方式使得模型训练更快、更不易发散。

简而言之，这张图生动地展示了 YOLOv2 如何将“凭空猜测一个框”这个复杂问题，分解为“选一个经验框（锚框），然后把它的中心和大小稍微调整一下”这个更容易学习和优化的问题。这正是 YOLOv2 在定位精度上取得显著提升的关键所在。

构：Passthrough Layer（直通层）。它的核心目的是融合不同尺度的特征，以提升模型（尤其是小目标）的检测能力。

图例解读与工作原理

1. 输入与拆分

- 输入：来自网络浅层的、尺寸较大的特征图，维度为 $26*26*512$ 。这个特征图分辨率高，包含丰富的空间细节和位置信息，但对目标的语义理解（是什么）较弱。

- 操作：对该特征图进行“拆分”。具体技术称为“空间重组”或“深度到空间”操作。它将 $26*26$ 的网格，每 $2*2$ 的相邻区域（共 4 个位置）的通道值拆开。

2. 双路处理

- 上通路（Passthrough）：将上述拆分出的 4 组通道值，在通道维度上进行拼接。这样，空间尺寸就缩小了（ $26/2=13$ ），但通道数增加了（ $512*4=2048$ ）。因此， $26*26*512$ 的特征图被重组为 $13*13*2048$ 。这个过程保留了浅层特征的精细空间信息，只是换了一种表示方式。

- 下通路（Pooling+Convolution）：同一张输入特征图也经过池化（下采样）和一系列卷积操作，得到维度为 $13*13*1024$ 的深层特征。这个特征图语义信息强，但空间细节粗。

3. 融合与输出

- 叠加：这里“叠加”指的是通道拼接。将上通路保

留细节的特征 ($13 \times 13 \times 2048$) 与下通路富含语义的特征 ($13 \times 13 \times 1024$) 在通道维度上进行拼接。

◦输出：最终得到一个 $13 \times 13 \times 3072$ （即 $2048+1024=3072$ ）的融合特征图。这个特征图同时具备了浅层的精细位置信息和深层的强大语义信息。

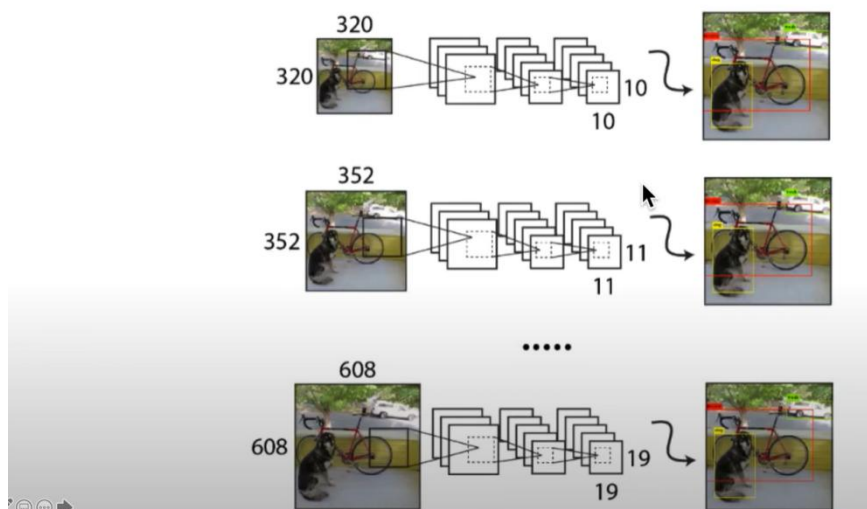
核心目的与优势

这种设计的精妙之处在于，它让后续的检测头（Detection Head）能够基于一个同时包含“在哪里”（细位置）和“是什么”（强语义）的信息富集特征图来进行预测。这对于检测小目标尤其有利，因为小目标在经历多次下采样后，在深层特征图上可能只剩下几个像素甚至消失，但其位置信息在浅层特征中非常清晰。

这也是 YOLOv2/v3 等单阶段检测器在保持速度优势的同时，提升检测精度（特别是小目标检测精度）的一项重要技术。

5. Multi-Scale Training:

Multi-scale training: +1.5% mAP



这张图非常直观地展示了 YOLOv2 中一项强大且简单的训练技巧：多尺度训练，以及它如何直接带来了+1.5% mAP 的性能提升。

图片纵向排列的三组内容，展示了在三种不同输入分辨率下，同一个 YOLOv2 模型的训练与检测过程：

1. 左侧列：输入图像

- 展示了网络在训练或测试时接收到的实际输入图像。其尺寸分别是 320×320 、 352×352 和 608×608 。

- 图像内容相同（都包含狗和自行车），但分辨率不同。

2. 中间列：特征图与预测网格

- 这代表了网络最后的特征图，YOLOv2 正是在这个尺度上进行目标预测。

- 关键数字 10、11、19 表示特征图被划分成的网格数量（即 $S \times S$ 中的 S ）。

◦核心关系：由于 YOLOv2 的网络结构是全卷积的（无全连接层），输入图像尺寸变化会导致最终特征图尺寸等比变化。具体来说，YOLOv2 的下采样总步长为 32。因此：

▪输入 $320 \times 320 \rightarrow$ 特征图尺寸为 10×10
($320/32=10$)

▪输入 $352 \times 352 \rightarrow$ 特征图尺寸为 11×11
($352/32=11$)

▪输入 $608 \times 608 \rightarrow$ 特征图尺寸为 19×19
($608/32=19$)

◦网格越密集（如 19×19 ），对图像的分割就越细，模型对小目标的定位能力理论上就越强。

3. 右侧列：检测结果

◦展示了模型在对应输入分辨率下的检测输出。可以看到，在更高分辨率（ 608×608 ）下，检测框（红色）更为精准。

多尺度训练的原理与价值

这张图背后的核心思想是 “动态多尺度训练”：

•如何操作：在训练过程中，不是固定使用一种输入尺寸，而是每隔一定的迭代次数（例如每 10 个批次），就随机选择一个新的输入尺寸（从 32 的倍数中随机选取，如 $\{320, 352, \dots, 608\}$ ）。

•为何有效：

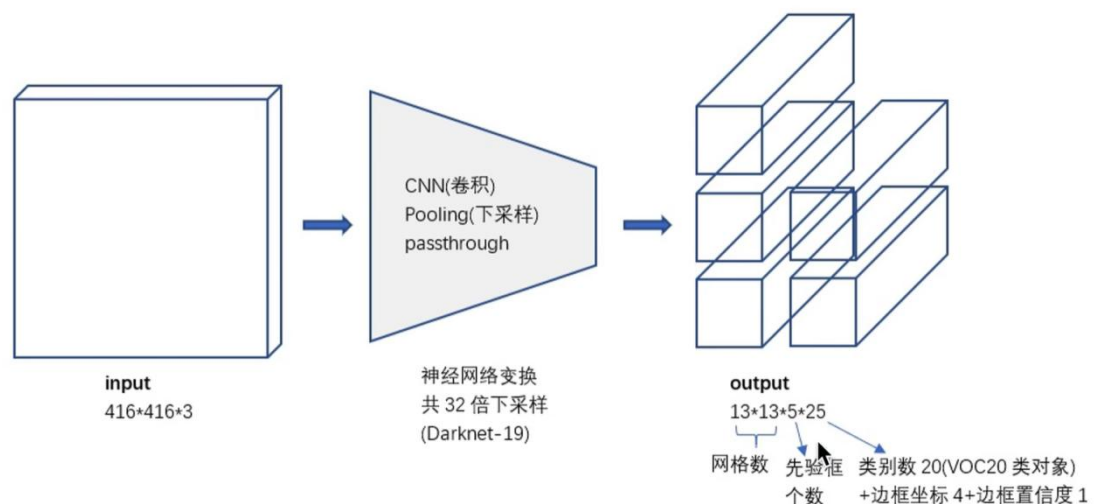
1. 增强模型鲁棒性：迫使网络学会在不同尺度下进行有效预测，从而能够更好地泛化到各种大小的输入图像。这相当于一种强有力的数据增强形式。

2. 同一模型，多种分辨率：训练好的同一个模型，可以在测试时根据需要灵活调整输入尺寸。

- 需要高速推理时：使用较小的输入（如 320×320 ），网络运行更快。

- 需要高精度检测时：使用较大的输入（如 608×608 ），网络能捕捉更多细节，尤其提升小目标检测精度。

3. 更好的正则化效果：不断变化的输入尺寸可以防止模型过拟合到某种固定的图像尺度上。



YOLOV3:

整体架构解析:

YOLOv3 的核心是 Darknet-53 骨干网络+多尺度检测头，整体流程如下：

1. 输入： $416 \times 416 \times 3$ 的 RGB 图像。

2. 骨干网络（Darknet-53）：

由一系列 CBL 和残差块（res1, res2, res8, res8, res4）堆叠而成，用于提取多尺度特征。

输出三个不同尺度的特征图，分别对应大、中、小目标的检测。

3. 多尺度特征融合：

最深层的特征图（ 13×13 ）经过 5 个 CBL 后，直接输出第一个检测头 y1，负责检测大目标。

同时，该特征图经过 CBL 和上采样后，与中层特征图（ 26×26 ）进行 Concat 融合，再经过 5 个 CBL 后输出第二个检测头 y2，负责检测中等目标。

融合后的特征图再次经过 CBL 和上采样，与最浅层特征图（ 52×52 ）Concat，再经过 5 个 CBL 后输出第三个检测头 y3，负责检测小目标。

输出：

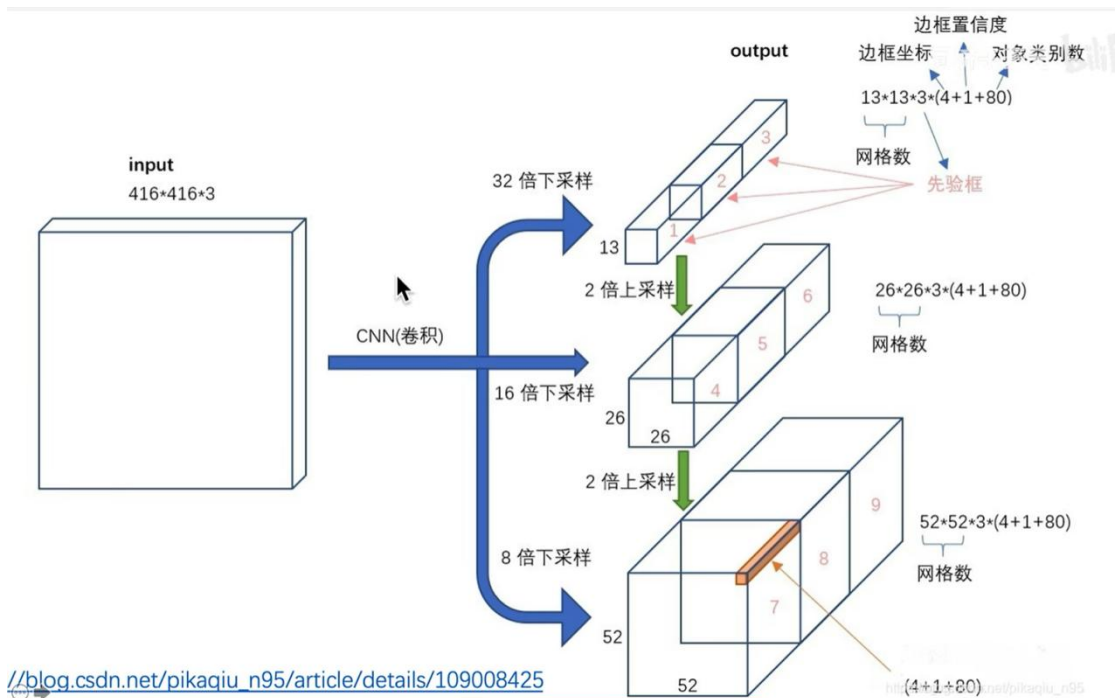
y1: 13×13 （下采样 32 倍） $\times 255$:感受野大，包含强语义信息，用于检测大目标。

y2: 26×26 （下采样 16 倍） $\times 255$:平衡细节和语义信息，用于检测中等尺寸目标。

y3: 52×52 （下采样 8 倍） $\times 255$:感受野小，保留丰富的细节信息，专门用于检测小目标。

其中 255 是由 3 个锚框 $\times$ (4 个坐标+1 个置信度+ 80 个

通过上采样（Upsample）和拼接（Concat）操作，将深层的语义信息与浅层的细节信息融合，极大提升了对不同尺寸目标，尤其是小目标的检测能力。



性能与特点

特性	描述	优势
速度与精度平衡	在COCO数据集上，YOLOv3在保持较高mAP精度的同时，可实现每秒数十帧的推理速度 ^{5 8} 。	非常适合需要实时处理的应用场景，如视频监控、自动驾驶
小目标检测能力	多尺度特征融合机制使其对小目标的检测性能相比YOLOv2有显著提升 ^{3 5} 。	解决了前代模型的一大短板，在复杂场景中更实用 ⁸ 。
灵活性	通过调整输入图像尺寸（如320×320, 416×416, 608×608），可以在速度和精度之间进行灵活权衡 ⁸ 。	可根据实际硬件和性能需求进行配置 ⁵ 。



原图大小为500×333



直接Resize为416×416



加灰条Resize为416×416

YOLO v3 目标检测损失函数 同济子豪兄

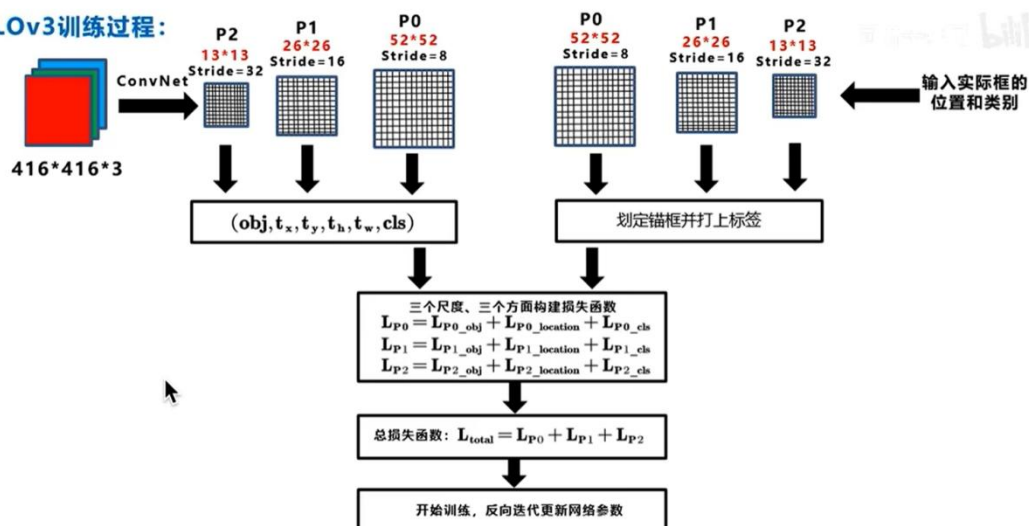
预测框分为三种情况：（不同代码实现细节可能不完全相同）

正例 positive 与GT IOU最大
负例 negative IOU<0.5
忽略 ignore IOU>0.5但非最大

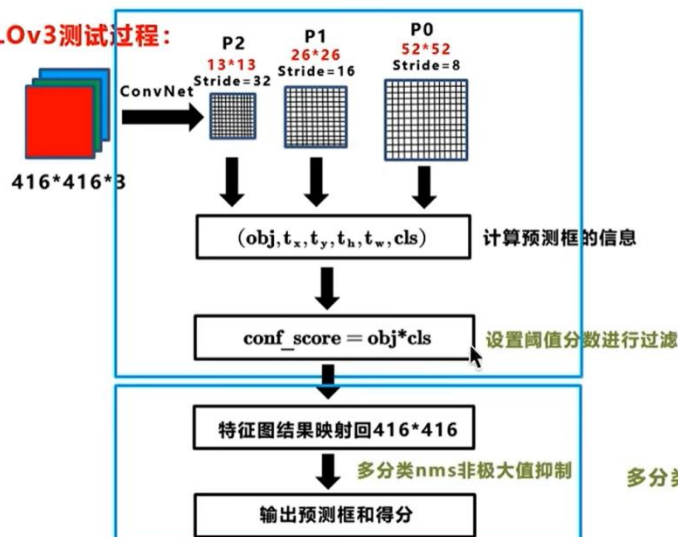
$$\begin{aligned}
 & \text{正样本坐标} \quad \lambda_{coord} \sum_{i=0}^{S^2} \sum_{j=0}^B 1_{i,j}^{obj} \cdot [(b_x - \hat{b}_x)^2 + (b_y - \hat{b}_y)^2 + (b_w - \hat{b}_w)^2 + (b_h - \hat{b}_h)^2] \quad \text{中心点坐标, 中心点宽高} \\
 & \text{正样本置信度和类别 (标签为1)} \quad + \sum_{i=0}^{S^2} \sum_{j=0}^B 1_{i,j}^{obj} \cdot [-\log(p_c) + \sum_{i=1}^n \text{BCE}(\hat{c}_i, c_i)] \quad \text{置信度标签为1, 类别数, 逐类别计算=交叉熵损失} \\
 & \text{负样本置信度 (标签为0)} \quad + \lambda_{noobj} \sum_{i=0}^{S^2} \sum_{j=0}^B 1_{i,j}^{noobj} \cdot [-\log(1 - p_c)] \quad \text{置信度标签为0} \\
 & \text{遍历所有预测框} \quad \text{输入416*416时, } S^2=13*13=169, B=3
 \end{aligned}$$

二元交叉熵损失函数 (Binary Cross Entropy)
 $BCE = -\hat{c}_i \log(c_i) - (1 - \hat{c}_i) \log(1 - c_i)$
 该是标签值 (非0即1) 该是预测值 (0-1之间)

YOLOv3训练过程:



YOLOv3测试过程:



第一部分:
(1) 计算预测框的信息

(2) 设置阈值, 过滤得分低的框

第二部分:

多分类nms消除各个类别重叠较大的预测框

YOLO (You Only Look Once) 系列算法本质上将目标检测问题视为一个单一的回归问题。这是 YOLO 区别于传统两阶段检测器 (如 Faster R-CNN) 的核心设计理念。

一、YOLO 的回归本质

YOLO 的设计思想是端到端的直接回归：

- 输入图像被均匀划分为 $S \times S$ 的网格。每个网格单元负责预测固定数量的边界框 (Bounding Boxes) 以及这些框的置信度 (Confidence Score) 和类别概率。

- 回归目标：模型直接通过卷积网络回归出边界框的坐标 (中心点 x 、 y ，宽度 w ，高度 h)、置信度 (表示框内是否存在目标及定位准确性) 和类别概率。

- 损失函数：YOLO 的损失函数通常结合了定位损失 (如均方误差或 IoU 损失)、置信度损失和分类损失，整体优化的一个多元回归任务。

二、YOLO 的演进与回归的强化

- YOLOv1：开创性地将检测定义为回归问题，但边界框预测不够稳定 (如对尺度敏感)。

- YOLOv2/v3：引入锚框 (Anchor Boxes) 作为先验，使回归目标从直接预测坐标变为预测相对于锚框的偏移量，提升了定位精度，但本质仍是回归 (预测偏移量而非生成提议)。

- 后续版本 (如 YOLOv4 及以上)：尽管引入了更复杂的

模块（如注意力机制、FPN），但核心仍是单阶段回归框架，即“一步到位”地预测框和类别。

三、与两阶段方法的对比

- 两阶段方法（如 Faster R-CNN）：先生成区域提议（Region Proposals），再对提议分类和微调边界框。这本质上是分类+回归的组合，而非纯回归。

- YOLO 的优势：回归策略使其速度极快，适合实时检测，但早期版本在精度上可能逊于两阶段方法（后续版本已大幅改善）。

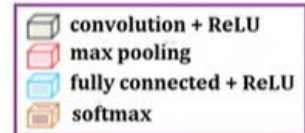
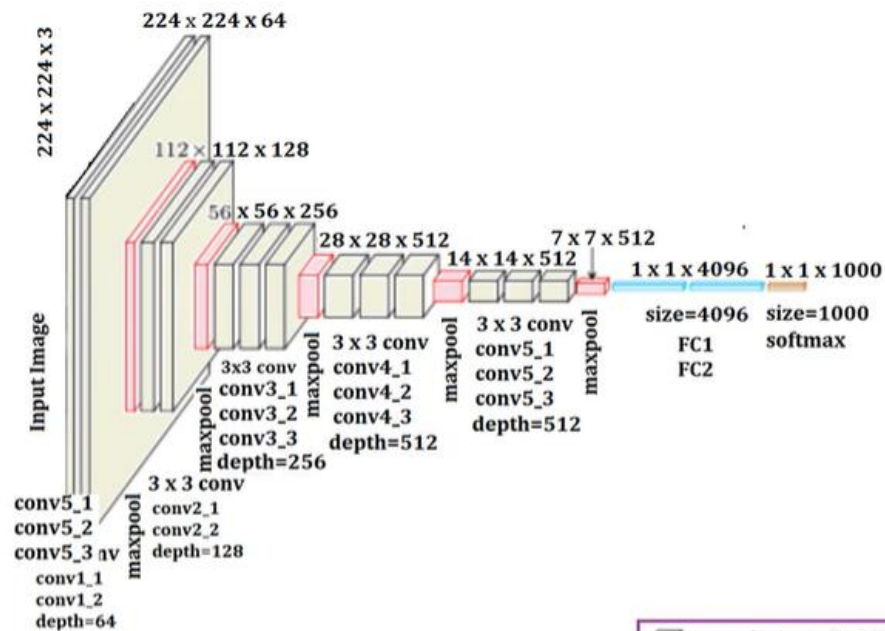
四、注意事项

- YOLO 不是“纯粹”的回归：它融合了分类任务（预测类别概率），但整体框架是通过回归直接输出检测结果。

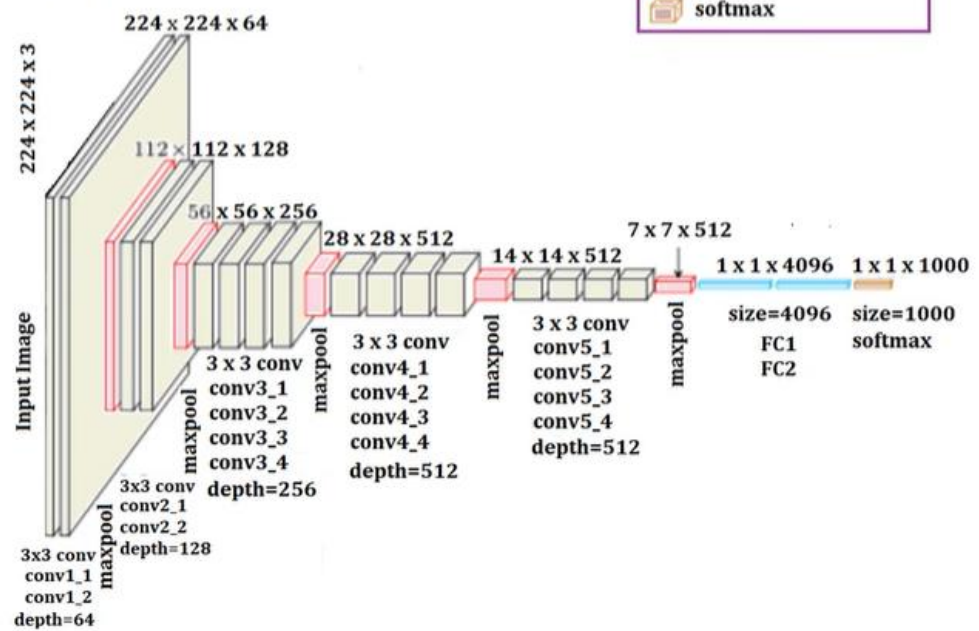
总结

YOLO 系列始终以回归为核心，通过单次前向传播直接预测目标位置和类别，实现了速度与精度的平衡。这种回归范式使其成为实时检测领域的标杆算法。

VGG16



VGG19



VGG-16

ImageNet

